People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

University of Science and Technology Houari Boumediene
Faculty of Computer Science
Department of Artificial Intelligence
Master's Thesis
Specialization:
Intelligent Computer Systems

Title:
Using Deep Reinforcement Learning for Automatic Code Optimization in the MLIR Compiler

---

Supervised by:
Dr. Riyadh Baghdadi

Jury Members:
Prof. Malika Ioualalen
Prof. Faiza Khellaf

Presented by:
Rafik Bouloudene
Djad Bouchama

Defense Date:
29/06/2025

Project: SII_09 / 2025

Acknowledgements

First and foremost, we express our deep gratitude to God Almighty for granting us the strength, patience, and perseverance necessary to carry out this work. It is through His mercy that we were able to progress through each stage of this journey with determination.

We would like to extend our sincerest thanks to our supervisor, *Mr R.BAGHDADI*, for his guidance throughout this project. His availability, scientific rigor, and the clarity of his advice greatly contributed to the progress of our work and the development of our skills.

We also extend our sincere gratitude to *Mrs. IOUALALEN* for agreeing to chair our jury, as well as to *Mrs. KHELLAF* for being a part of it, and for the honor they do us by accepting to evaluate our work.

"I wish to express my deep gratitude to my mother *Lamia* for her unconditional love and patience, to my father *Mokhtar* for his constant support, as well as to my brother *Walid* and my sister *Manel*, whose encouragement has accompanied me throughout this journey. Their presence has been essential during both the most difficult and most rewarding moments." *Rafik Bouloudene*

"I dedicate this work to my dear parents, whose unwavering support and constant encouragement have known no bounds throughout the realization of this work—and long before. Their words, their actions, their care, and even their silences have been a vital foundation that enabled me to successfully pursue my academic journey, and in particular this thesis. Their help has been crucial to my being where I am today.

I also extend this dedication to my two brothers and my sister, loyal companions at every step of my life. Their presence, listening, and support have played a decisive role in bringing this journey to completion." *Djad Bouchama*

Abstract

Code optimization at the compiler level is essential for improving the performance of machine learning models, especially as these models grow in size and computational complexity. Traditionally, such optimization has relied on expert-designed heuristics, which are often rigid, time-consuming to tune, and difficult to scale across architectures. Reinforcement Learning (RL) presents a compelling alternative, offering the ability to automatically discover optimization strategies through trial and error. However, applying RL to compiler infrastructures introduces several challenges, including the design of effective state representations, scalable action spaces, and robust learning environments.

This work presents a reinforcement learning framework for optimizing full neural network programs within the Multi-Level Intermediate Representation (MLIR) compiler infrastructure. We extend an existing RL agent to support realistic end-to-end programs by incorporating a structured observation space based on Abstract Syntax Trees (ASTs) and by introducing a hierarchical action space that includes both transformation and code navigation operations.

To support training and experimentation, we developed a complete RL environment integrated with MLIR. By shifting focus from isolated operations to full-program optimization, our approach highlights the potential of RL-guided compilation within a modern intermediate representation framework and establishes a foundation for future improvements in this space.

Key words: Reinforcement Learning, Code Optimization, MLIR, Compilers

Résumé

L'optimisation de code au niveau du compilateur est essentielle pour améliorer les performances des modèles d'apprentissage automatique, d'autant plus que ces modèles gagnent en taille et en complexité de calcul. Traditionnellement, cette optimisation repose sur des heuristiques conçues par des experts, lesquelles sont souvent rigides, longues à ajuster et difficiles à transposer d'une architecture à l'autre.

L'Apprentissage par Renforcement (AR) représente une alternative prometteuse, offrant la capacité de découvrir automatiquement des stratégies d'optimisation par essais et erreurs. Cependant, l'application de l'AR aux infrastructures de compilation présente plusieurs défis, notamment la conception de représentations d'état efficaces, d'espaces d'actions extensibles et d'environnements d'apprentissage robustes.

Ce travail présente un framework d'apprentissage par renforcement pour l'optimisation de programmes de réseaux de neurones complets au sein de l'infrastructure de compilation Multi-Level Intermediate Representation (MLIR). Nous étendons un agent AR existant pour prendre en charge des programmes de bout en bout réalistes en incorporant un espace d'observation structuré basé sur les Arbres Syntaxiques Abstraits (AST) et en introduisant un espace d'actions hiérarchique qui inclut à la fois des opérations de transformation et de navigation dans le code.

Pour permettre l'entraînement et l'expérimentation, nous avons développé un environnement AR complet, intégré à MLIR. En déplaçant l'attention des opérations isolées vers l'optimisation de programmes complets, notre approche met en évidence le potentiel de la compilation guidée par l'AR au sein d'un framework de représentation intermédiaire moderne et jette les bases pour de futures améliorations dans ce domaine.

Mots-clés : Apprentissage par Renforcement, Optimisation de Code, MLIR, Compilateurs

Contents

List of Figures

List of Tables

General Introduction

Code optimization is a critical area in computer science that aims to improve program performance by reducing execution time and resource usage. With the growing complexity of hardware architectures and machine learning workloads, traditional rule-based optimization techniques are increasingly limited in their ability to adapt. Reinforcement Learning (RL) has emerged as a promising automated approach for program optimization. By learning from experience through trial and error, RL agents can discover sequences of transformations that improve performance without explicit supervision. This approach has shown impressive results in domains such as game playing and robotics, and is now being explored in compiler optimization.

Applying RL to compilers presents exciting opportunities but also several challenges. The search space of possible optimizations is vast, the structure of programs is highly variable, and designing meaningful state and action representations is complex. An RL agent must be able to perceive the relevant features of a program to make informed transformation decisions. Moreover, the diversity of available transformations and their interaction effects add significant difficulty to the learning process. Despite these challenges, recent advances have shown that RL can successfully learn optimization strategies that outperform hand-crafted heuristics.

This work explores the application of RL to compiler optimization in the context of the MLIR (Multi-Level Intermediate Representation) framework. Unlike prior work focused on isolated operations, our approach aims to optimize full neural network programs by extending an existing RL agent. Specifically, we expand the agent's observation and action spaces to support complex transformations such as operation fusion and code navigation. Our objective is to demonstrate that an RL-based optimizer can efficiently explore the transformation space and achieve meaningful performance improvements on real workloads.

This thesis is structured into three main parts. The first part introduces the background and related work on code optimization, reinforcement learning, and the MLIR infrastructure. The second part describes the design of our extended RL agent, including its environment, action set, and observation model. The final part presents experimental results on benchmark models, evaluates the performance gains obtained through our method, and discusses the limitations and future directions of RL-based compilation in MLIR.

Chapter 1

Code Optimization

1.1 Introduction

Code optimization, particularly in the context of modern compiler design, represents a fundamental pillar of computer science that bridges the gap between human-written programs and efficient machine execution. As software systems grow in complexity and diverse hardware architectures continue to emerge, the role of compiler optimizations becomes increasingly critical in delivering high-performance, resource-efficient code.

At its core, compiler optimization is a sophisticated interplay between preserving program semantics and enhancing execution efficiency. Modern compilers must navigate complex trade-offs between optimization opportunities while ensuring that the original program's behavior remains unchanged. This challenge has evolved from relatively straightforward transformations to a multi-dimensional problem that must consider various factors including execution time, memory usage, power consumption, and hardware-specific characteristics.

This chapter examines the evolution of compiler optimization, from fundamental principles to cutting-edge implementations. Beginning with traditional techniques like control-flow modifications and loop transformations, we explore how these foundations have adapted to modern hardware capabilities. The discussion progresses to contemporary challenges in compiler development, including the complexities of optimization phase ordering and hardware-specific constraints.

We conclude by examining the MLIR (Multi-Level Intermediate Representation) infrastructure, a modern framework that exemplifies the field's response to increasingly diverse computing environments. Through this progression, we bridge classical optimization theory with practical implementation challenges in contemporary compiler design.

1.2 Fundamentals of Compiler Optimization

Compiler optimization is a crucial aspect of the software development process, aiming to improve the performance and efficiency of generated code. This section outlines the fundamental principles of compiler optimization, including the preservation of semantics, optimization goals, and key trade-offs involved in the optimization process.

1.2.1 Semantic Preservation

One of the primary objectives of compiler optimization is to enhance performance without altering the intended behavior of the program. This requirement is known as semantics preservation (Bacon et al. 1993). It ensures that optimized code produces the same output as the original code for all possible inputs. Achieving this involves rigorous analysis and transformation of code while maintaining its logical structure and functional correctness. Various techniques, such as constant propagation and dead code elimination, are employed to optimize code while ensuring that the semantics remain intact.

1.2.2 Optimization Goals

Compiler optimization encompasses multiple objectives aimed at improving the efficiency and performance of the generated code. These goals go beyond merely reducing execution time to include broader considerations such as resource utilization, portability, and maintainability (Aho et al. 2006). The following are the primary optimization goals:

- **Performance Improvement**: The primary aim is to enhance the speed and responsiveness of applications by reducing execution time. Key strategies include:
  - Minimizing instruction counts to streamline execution.
  - Optimizing memory access patterns to reduce latency, especially in data-intensive operations.
  - Exploiting parallel execution capabilities to fully utilize multi-core and vector processing architectures.
- **Resource Utilization**: Efficient use of system resources, such as CPU cycles, memory bandwidth, and cache, is critical for achieving high-performance computing. This involves:
  - Reducing memory footprint to enable better scalability in memory-constrained environments.
  - Improving cache locality to minimize cache misses, thereby accelerating memory access.
- **Code Size Optimization**: In some contexts, such as embedded systems or mobile applications, minimizing the size of compiled binaries is crucial. Techniques used to achieve this goal include:
  - Dead code elimination to remove unnecessary or unreachable code, streamlining the final executable.
  - Leveraging code compression techniques and compact data representations for storage-constrained environments.
- **Portability and Scalability**: Ensuring that code performs efficiently across a wide range of platforms and hardware configurations. This involves designing optimizations that adapt to specific architecture features, such as vector units, GPUs, or distributed systems.

Modern advancements in compiler optimization increasingly incorporate adaptive techniques, including machine learning, to dynamically tailor memory management and execution strategies based on observed runtime behavior. These context-aware optimizations offer significant improvements in both performance and efficiency.

1.2.3 Key Trade-offs

Compiler optimization often involves trade-offs that must be carefully considered to achieve the best balance between performance, resource usage, and compilation efficiency. These trade-offs arise because optimizing for one aspect of a program can negatively impact another. For instance, increasing execution speed through aggressive optimizations may lead to larger binary sizes, which can be problematic in memory-constrained environments. Additionally, the complexity of modern software systems means that a single optimization can have cascading effects on other parts of the code, making it crucial for compilers to evaluate the overall impact of each optimization strategy. Developers must also consider the implications of these trade-offs on the maintainability and portability of their code. As such, understanding these trade-offs is essential for making informed decisions about which optimizations to apply based on the specific requirements and constraints of the target application.

- **Time vs. Space**: Many optimizations that improve execution speed may increase memory usage (and vice versa). For example, loop unrolling can enhance performance by reducing loop overhead but may result in larger code size.
- **Compile Time vs. Runtime Performance**: Some optimizations require significant compile-time analysis and transformations that can delay build times. Striking a balance between fast compilation and optimized runtime performance is essential.
- **Generalization vs. Specialization**: Generalized optimizations may work well across a range of programs but might not achieve optimal performance for specific cases. Conversely, specialized optimizations tailored to particular applications can yield better results but may reduce portability and reusability.

1.3 Traditional Optimization Techniques

Traditional optimization techniques in compiler design can be broadly categorized into two types: **static** and **dynamic optimizations**. Each technique has its strengths and weaknesses, and they are applied based on the specific requirements of the program being compiled.

1.3.1 Static vs. Dynamic Optimization

Static optimization occurs during the compilation phase, where the compiler analyzes the code without executing it. This approach allows for optimizations based on program structure and data flow analysis. Examples include constant folding, dead code elimination, and loop invariant code motion. However, static optimization may not account for runtime behavior, which can lead to suboptimal performance in certain scenarios.

Dynamic optimization, on the other hand, takes place during program execution. It leverages runtime information to make adjustments to the code, enabling more informed optimizations based on actual usage patterns. Techniques such as just-in-time (JIT) compilation and adaptive optimization fall under this category. While dynamic optimization can yield significant performance improvements by tailoring optimizations to real-time data, it often incurs overhead due to the need for monitoring and analysis during execution. (Bhattacharjee 2024)

Within the realm of static optimizations, two key areas of focus are control-flow optimizations, which refine the execution logic of programs, and data-flow optimizations, which analyze and optimize the flow of data along different execution paths. Both rely heavily on compile-time analysis to eliminate redundancies, reduce overhead, and improve overall program performance. The following sections delve into these techniques in greater detail.

1.3.2 Control-Flow Optimizations

Control-flow optimizations focus on improving the efficiency of how a program executes its control structures. Below are key techniques used in this category:

Dead Code Elimination: Dead code elimination removes parts of the code that do not affect the program's output. These could be statements that are never executed or computations whose results are not used. By eliminating such code, compilers can improve both performance and code readability.

Inlining: Inlining replaces function calls with the actual function code to eliminate the overhead of invoking functions. This can improve execution speed, especially for small functions that are called frequently.

Figure 1.1 illustrates the code inlining optimization. In this example, the function call is replaced with the function's body, eliminating the overhead of the function call and allowing the computation to be performed directly within the calling context.

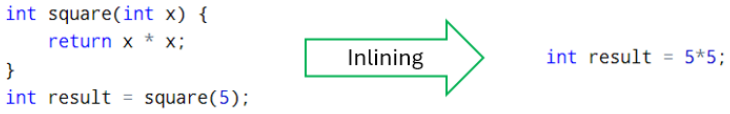


Figure 1.1: Example of a code inlining optimization

Code Motion: Code motion moves computations outside of loops when their results do not change within the loop's iterations. This transformation reduces repeated calculations and improves performance by avoiding redundant computations.

Figure 1.2 illustrates the code motion optimization. In this example, `limit - 2` is identified as a loop invariant, meaning its value does not change during the loop iterations. By applying code motion, this computation is moved outside the loop, so it is calculated only once instead of being redundantly recalculated in every iteration.

Figure 1.2: Code motion optimization

By employing these techniques, control-flow optimizations improve the efficiency and performance of programs by reducing unnecessary computations and execution overhead.

1.3.3 Data-Flow Optimizations

Data-flow optimizations enhance the handling of data dependencies within a program by analyzing how data flows through different execution paths. These optimizations rely on data-flow analysis, a set of techniques that provides insight into how data is computed and propagated across the program (Aho et al. 2006). By understanding the flow of data, compilers can identify and eliminate inefficiencies such as redundant calculations or unused assignments. Key techniques that depend on data-flow analysis include:

- **Constant Propagation**: Replacing variables that hold constant values with their actual constants to simplify expressions.

- **Common Subexpression Elimination**: Identifying and eliminating duplicate calculations by storing results of previously computed expressions.

By optimizing data flow, these techniques help reduce computation time and resource consumption.

1.4 Loop Optimizations

Loop optimizations are critical in compiler design due to the significant amount of execution time that programs spend in loops. By optimizing loops, compilers can enhance performance, reduce overhead, and improve cache utilization. Efficient loop execution is essential for applications, especially in scientific computing and data-intensive tasks, where loops often dominate runtime.

This section explores various loop optimization techniques, including their importance and specific methodologies.

1.4.1 Loop Parallelization

Loop parallelization is an optimization technique that enables simultaneous execution of independent iterations of a loop. This is typically achieved by dividing the loop's workload into smaller, parallel tasks that can be executed on multiple processors or cores. The primary goal is to reduce the overall execution time by leveraging multi-core or multi-threaded systems to execute computations in parallel (Bacon et al. 1993). For loops with independent iterations, parallelization can significantly increase performance by making full use of available hardware resources.

Benefits of Loop Parallelization:

- **Improved Performance**: Increased throughput and reduced execution time, particularly for computationally intensive tasks, by executing iterations concurrently.
- **Efficient Resource Utilization**: Better utilization of hardware resources by engaging multiple processors or cores, reducing idle time.
- **Scalability**: Adaptable to different hardware configurations, performance improves as the number of processors or cores increases.

1.4.2 Loop Unrolling

Loop unrolling is an optimization technique used to enhance loop performance by reducing loop overhead and increasing instruction-level parallelism. This is achieved by transforming a loop that executes n iterations into one that executes $\frac{n}{r}$ iterations, where r is the unroll factor (Fog 1996). Each iteration in the unrolled loop performs r calculations as shown in figure 1.3, reducing the frequency of loop control operations such as incrementing counters and checking conditions. This technique is most effective when n is divisible by r, ensuring seamless execution without the need for additional handling of remainder iterations.

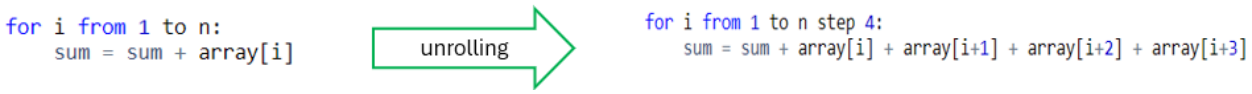


Figure 1.3: Example of a loop unrolling with a factor of 4

Benefits of Loop Unrolling:

- **Reduced Loop Overhead**: Fewer loop control instructions are executed (e.g. loop counter increment and condition check).
- **Increased Instruction-Level Parallelism**: More instructions can be executed in parallel by the processor.
- **Improved Cache Locality**: By accessing memory in a more linear pattern, cache performance can be improved.

1.4.3 Loop Skewing

Loop skewing is an optimization technique that adjusts the bounds and indices of nested loops to resolve dependencies, enabling parallel execution of loop iterations (Laforest 2010). By introducing a skew factor, this approach shifts the dependency patterns, increasing the distance between dependent iterations and making the loops amenable to parallelization while preserving program correctness. Loop skewing is particularly effective in computationally intensive tasks with inter-loop dependencies.

Figure 1.4 demonstrates loop skewing applied to a nested loop. The original loop executes iterations independently, while the skewed version adjusts the inner loop bounds (`j` from `i+1` to `n`) and reindexes array accesses (`j` replaced with `j-i`). This transformation aligns iterations to facilitate parallelization or optimize memory access.

Figure 1.4: Example of loop skewing applied to a nested loop.

Benefits of Loop Skewing:

- **Dependency Resolution**: Resolves dependencies in nested loops, making parallel execution feasible.
- **Improved Performance**: Enhances execution speed by enabling parallelization in previously sequential loops.
- **Scalable Optimization**: Adapts well to various hardware configurations, maximizing resource utilization.

1.4.4 Loop Tiling

Loop tiling, also known as blocking, is an optimization technique that divides a loop into smaller blocks or tiles, which can be executed independently (Bacon et al. 1993). The goal is to improve memory access patterns, enhance cache locality, and increase parallelism by processing sub-portions of data at a time, rather than the entire data set. This technique is especially useful for multidimensional loops, such as matrix multiplication or convolution operations.

Figure 1.5 illustrates loop tiling with a tile size of 64. The outer loops (`TI` and `TJ`) iterate over tiles, while the inner loops (`i` and `j`) handle computations within each tile.

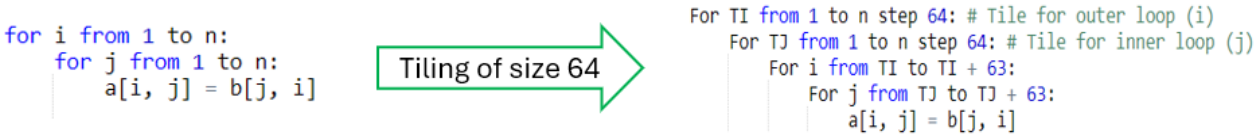


Figure 1.5: Example showing how loop tiling affects the loops structure

Benefits of Loop Tiling:

- **Improved Cache Locality:** By dividing the loop into smaller blocks that fit in the cache, tiling ensures that data accessed during computation remains in cache longer, reducing memory latency.
- **Reduced Memory Access:** Tiling minimizes the number of times data needs to be loaded from main memory, as smaller blocks reuse data more effectively within the cache.
- **Enhanced Parallelism:** Tiling can improve parallelism by allowing blocks to be processed concurrently on multiple cores, further enhancing performance.

1.4.5 Loop Inversion

Loop inversion, also referred to as loop reversal, is an optimization technique that changes the direction in which a loop traverses its iteration range as shown in figure 1.6. This transformation is particularly useful for reordering dependence vectors, enabling optimizations when combined with other iteration space transformations (Bacon et al. 1993). By inverting the loop, the iteration variable counts downwards instead of upwards, which can simplify certain hardware instructions and reduce loop overhead on specific architectures.

Figure 1.6: An example that illustrate a loop inversion

Benefits of Loop Inversion:

- **Dependence Vector Adjustment**: Modifies the dependence structure to enable further optimizations in nested loops.
- **Reduced Loop Overhead**: Eliminates the need for compound comparison instructions, particularly on architectures without native support for them.
- **Memory Optimization**: Can minimize the need for temporary arrays when working with specific array operations or programming models.

1.4.6 Loop Unswitching

Loop unswitching is an optimization technique that moves conditional statements out of a loop when the condition is invariant (i.e., does not depend on the loop variable). This transformation eliminates the need to evaluate the condition repeatedly inside the loop body, improving performance by simplifying the loop and enabling further optimizations like parallelization. (Bacon et al. 1993)

In loop unswitching, the original loop is split into multiple independent loops, one for each branch of the conditional. While this may increase code size due to loop duplication, it reduces runtime overhead, improves CPU pipelining efficiency, and can expose opportunities for parallel execution.

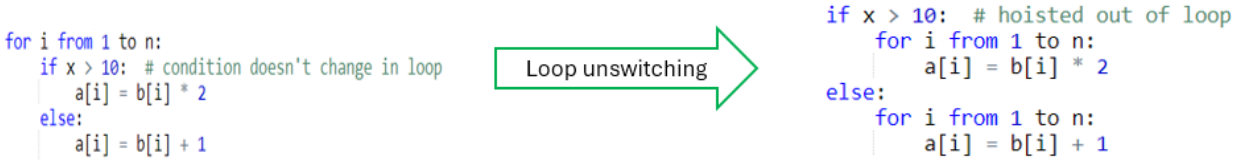


Figure 1.7: Example showing loop unswitching applied to a loop

Figure 1.7 demonstrates loop unswitching by hoisting an invariant condition `x > 10` outside the loop, creating two specialized loops and eliminating repeated condition checks.

Benefits of Loop Unswitching:

- **Reduced Overhead**: Eliminates repeated conditional evaluations within the loop, improving efficiency.
- **Improved Pipelining**: Optimizes the loop for CPU pipelines by removing branch instructions within the loop body.
- **Exposes Optimizations**: Facilitates other loop transformations by simplifying the control flow.

1.4.7 Loop Interchange

Loop interchange is an optimization technique used to improve data locality and loop performance by swapping the nesting order of loops. This transformation rearranges the iteration order to enhance cache usage and reduce memory access overhead (Fog 1996). By prioritizing the loop that accesses memory in a more linear pattern, the technique minimizes cache misses and improves execution efficiency.

This optimization is particularly beneficial for multi-dimensional arrays, where the default memory layout may not align with the access pattern of the original loop structure.

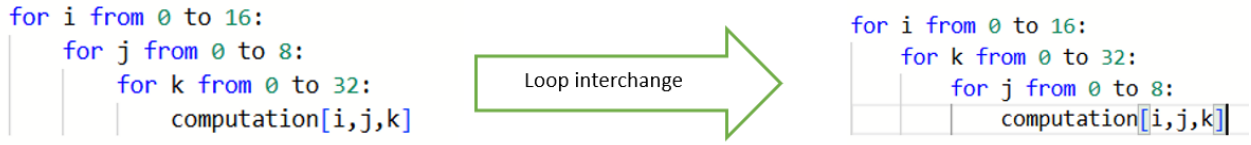


Figure 1.8: Example showing loop interchange

Benefits of Loop Interchange

- **Improved Cache Locality**: Reordering loops ensures that memory is accessed in a sequential manner, reducing cache misses.
- **Enhanced Performance for Multi-Dimensional Arrays**: Optimized access patterns align better with the memory layout, reducing the time spent on memory fetches.
- **Potential for Parallelization**: The new loop order may expose opportunities for loop-level parallelism.

1.4.8 Loop Vectorization

Loop vectorization is a compiler optimization technique designed to leverage the Single Instruction, Multiple Data (SIMD) capabilities of modern processors. It transforms sequential loops into vector operations, allowing for the simultaneous processing of multiple data elements. By grouping multiple elements into a vector, the same operation can be applied to all elements in parallel, significantly improving performance. This technique has been widely adopted in high-performance computing to enhance the execution of data-intensive operations, such as scientific simulations, image processing, and machine learning tasks (Bacon et al. 1993).

Benefits of Loop Vectorization

- **Increased Parallelism**: By processing multiple data elements simultaneously, vectorization can significantly boost performance in tasks with repetitive operations.
- **Better Utilization of Hardware Resources**: Vectorization allows processors to make better use of their SIMD units, improving throughput.
- **Reduction in Execution Time**: Parallel execution of operations results in reduced overall execution time for data-intensive applications.

1.4.9 Loop Fusion

Loop fusion is a critical compiler optimization technique that combines multiple sequential loops iterating over the same data structure into a single, more efficient loop (Bacon et al. 1993). By merging loops that perform related computations, this approach reduces loop overhead, improves memory locality, and enhances cache performance. The technique allows compilers to decrease the number of loop control instructions, create more contiguous memory access patterns, and ultimately lead to faster program execution by maximizing the utilization of processor resources and minimizing computational redundancies.

Benefits of Loop Fusion

- **Improved Cache Locality**: Fusing loops often improves memory access patterns, leading to fewer cache misses and better cache utilization.
- **Reduced Loop Overhead**: By combining multiple loops into one, the number of loop control instructions is reduced.
- **Increased Instruction-Level Parallelism**: Fusion allows for more opportunities for parallel execution, especially in multi-core processors.

These loop optimizations are critical for maximizing performance in applications with significant iterative processing requirements.

By employing these traditional optimization techniques, compilers can significantly enhance the efficiency and performance of generated code while balancing trade-offs related to compile time, runtime efficiency, and resource utilization.

1.5 Challenges in Modern Compiler Optimization

Compiler optimization faces several challenges that arise from the increasing complexity of modern hardware, the vastness of the optimization space, and the limitations of traditional methods. These challenges underline the need for innovative approaches, such as reinforcement learning, to overcome these hurdles.

1.5.1 The Phase Ordering Problem

The phase ordering problem has been a longstanding challenge in compiler optimizations, as the sequence of optimization phases applied to a program can lead to significant variations in performance (Wang et al. 2024).

Determining an optimal order for these optimization passes is computationally complex due to the interdependent nature of many optimizations. Some passes enable or amplify the effect of others, while certain combinations can counteract earlier improvements. The sheer number of potential phase orderings, growing exponentially with the number of passes, exacerbates the difficulty of solving this problem effectively in practice.

1.5.2 Hardware Complexity

Modern hardware architectures, such as multi-core CPUs, GPUs, and specialized accelerators, introduce complex behaviors that are difficult to model and optimize for. Features like caching, pipelining, and vectorization require hardware-specific tuning to achieve optimal performance. The rapid evolution of hardware technologies further exacerbates the problem, as traditional heuristics often fail to generalize across different architectures.

1.5.3 Dimensionality Problem

The optimization space for compilers is vast, with multiple parameters such as program input sizes, hardware configurations, and optimization flags. Exploring all combinations to find the best strategy is computationally infeasible due to the exponential growth of possibilities. Additionally, optimization decisions in one part of the code can have unintended consequences elsewhere, making it difficult to generalize effective strategies across programs.

These challenges highlight the need for adaptive and data-driven approaches, such as reinforcement learning, which can dynamically learn optimization strategies and navigate the large search space of possible optimizations. Additionally, frameworks like MLIR are instrumental in addressing hardware-specific features.

1.6 MLIR

MLIR (Multi-Level Intermediate Representation) is an advanced compiler infrastructure designed to address the growing complexity of modern computing systems. It provides a unified framework for representing computations at various abstraction levels, from high-level operations to low-level machine instructions. Developed as part of the LLVM project, MLIR aims to offer flexibility in optimizing diverse applications, particularly in machine learning and domain-specific languages (C. Lattner et al. 2021).

1.6.1 Core Features

- **Extensibility through Dialects:** One of MLIR's key strengths is its ability to define new operations, types, and attributes through dialects. This feature allows developers to create specialized languages for specific domains, making it easier to handle unique computational patterns.
- **Multi-Level Abstraction:** MLIR supports multiple layers of abstraction, which enables fine-grained optimizations. These levels span from higher-level algorithms to low-level hardware instructions, allowing for tailored optimizations that target performance bottlenecks across various stages of execution.

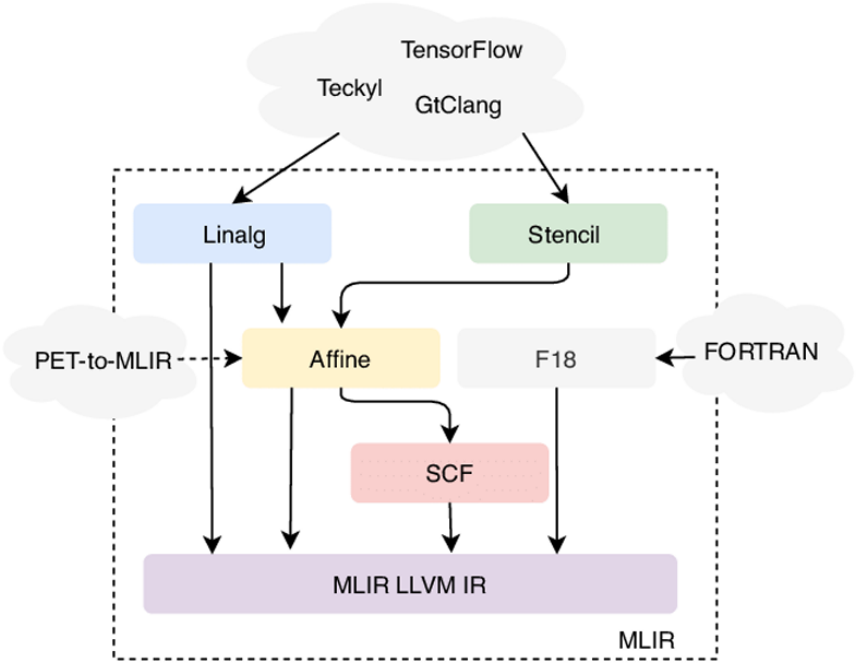


Figure 1.9: A set of MLIR dialects

- **Unified Optimization Framework:** MLIR integrates various optimization techniques within a cohesive framework. This enables reusability of optimization passes across different domains, improving efficiency in terms of both computation and development.

1.6.2 Real-World Application

MLIR's design is particularly beneficial in the realm of machine learning, where complex models require efficient representation and optimization. By providing a framework for targeted optimizations, MLIR facilitates faster model compilation and execution, leading to improved performance across heterogeneous hardware platforms. For example, the Tensor dialect allows MLIR to efficiently represent tensor computations, commonly used in deep learning frameworks. Other machine learning-specific dialects, such as the Linalg dialect for linear algebra operations, further optimize the processing of mathematical operations on hardware like GPUs and TPUs. Figure 1.9 illustrates the various MLIR dialects and how they interact with one another through a progressive lowering process, starting from high-level representations down to `MLIR LLVM IR`.

Conclusion

This chapter has examined the fundamental principles of compiler optimization, including key techniques such as control-flow, data-flow, and loop optimizations. These traditional methods have been widely adopted to enhance program performance. However, modern compilers face significant challenges, including the phase ordering problem, hardware complexity, and the dimensionality of the optimization search space. Addressing these issues requires adaptive, data-driven approaches like reinforcement learning and advanced frameworks such as MLIR.

Chapter 2

Reinforcement Learning

2.1 Introduction

Reinforcement Learning (RL) stands as a distinct and powerful technique within the field of machine learning, focused on training intelligent agents to make optimal decisions through interaction with an environment. Unlike supervised learning, which relies on labeled data, or unsupervised learning, which seeks hidden structures, RL is fundamentally about learning from consequences. An agent learns a desired behavior by receiving numerical rewards or penalties for its actions, gradually discovering a strategy, or "policy," that maximizes its cumulative reward over time. This trial-and-error approach mirrors how humans and animals learn and has proven remarkably effective for solving complex sequential decision-making problems in domains ranging from robotics and autonomous systems to game playing and resource management.

This chapter provides a comprehensive overview of the principles and methodologies that define Reinforcement Learning. We will begin by situating RL within the broader landscape of machine learning, highlighting its unique characteristics, such as the critical trade-off between exploration and exploitation. We will then deconstruct the RL framework into its core components—the agent, environment, actions, and rewards—and introduce the key mathematical concepts that underpin its algorithms, including value functions, policies, and environment models.

Following this foundational overview, we will explore the main categories of RL algorithms, distinguishing between model-based and model-free, value-based and policy-based, and on-policy and off-policy methods. To make these concepts concrete, we will conduct a deep dive into Q-learning, a seminal model-free algorithm. Finally, we will bridge the gap to modern techniques by introducing Deep Reinforcement Learning (DRL), discussing how deep neural networks are used as powerful function approximators. We will conclude with a detailed examination of state-of-the-art architectures, including Deep Q-Networks (DQN), REINFORCE, and Proximal Policy Optimization (PPO), which is the algorithm used in this work.

2.2 Machine Learning Foundations

Machine learning (ML) is a subset of artificial intelligence (AI) that focuses on developing systems capable of learning from data efficiently while also improving their learning without being explicit programmed to do so. By leveraging various statistical and analytical techniques using computational power, ML algorithms identify patterns and make data-driven predictions or decisions.This field has seen a rapid growth, with the availability of vast datasets, advancements in computational resources, and its applications in diverse domains such as healthcare, finance, autonomous systems, and natural language processing (Mitchell 1997). The versatility of machine learning lies in its ability to adapt to a wide range of tasks, making it a cornerstone of modern technological innovation.

2.2.1 Types of Machine Learning

- **Supervised Learning**

  Training a model using labeled data, where each input is associated with a known output, is the act of **supervised learning** in which the algorithm learns to map inputs to outputs by minimizing a given error function representing the error between its predictions and the actual labels - often called the *loss* function $J_\theta(X)$ -. This type of learning is widely used in tasks such as **classification** (applications in which the aim is to assign each input to a number of discrete categories) - **e.g.,** spam detection - and **regression** (if the desired output consists of one or more continuous variables) - **e.g.,** predicting housing prices - . Supervised learning is particularly effective when a large dataset with accurate labels is available, making it a cornerstone of many modern machine learning applications (Bishop 2006).

- **Unsupervised Learning**

  Unsupervised learning (learning without a teacher) focuses on analyzing and interpreting data without labeled outputs that steer the models toward the correct answer.

  > The goal is to directly infer the properties of the dataset's probability density without the help of a supervisor or a teacher providing correct answers or degree-of-error for each observation.(Hastie et al. 2009)

  It aims to uncover hidden patterns, relationships, or structures within the data.Techniques such as clustering (e.g., grouping customers based on purchasing behavior) and dimensionality reduction (e.g., principal component analysis for feature selection) are prominent examples. Since it does not rely on labeled data, unsupervised learning is valuable in exploratory data analysis and scenarios where labeling is impractical, expensive, or simply unavailable.

- **Reinforcement Learning**

  Reinforcement learning is based on an agent interacting with an environment and learning what to do so as to maximize a numerical reward signal.(Richard S Sutton et al. 2018). This approach models decision-making processes, where the agent receives feedback in the form of rewards or penalties. Applications include robotics, where agents learn motor control, and games (prominent example include AlphaGo (Silver et al. 2016) an RL Agent that was able to defeat a Go world champion), where agents develop complex strategies. Reinforcement learning excels in dynamic environments where explicit instruction is unavailable The learner (i.e. the agent) is not told which actions to take, but instead must discover which actions yield the most reward by trying them (Richard S Sutton et al. 2018), making it a powerful method for real-world problems.

2.2.2 Why RL is Different

It might seem that the terms supervised learning and unsupervised exhaustively classify machine learning paradigms, in reality this is not the case. While reinforcement learning does not rely on labeled data to judge a model's output, which would lead some to believe that it belongs to unsupervised learning, at the same time it does not try to infer hidden structure. In reality, reinforcement learning is neither, it is trying to maximize a reward signal collected by the agent.

> We therefore consider reinforcement learning to be a third machine learning paradigm, alongside supervised learning and unsupervised learning and perhaps other paradigms. (Richard S Sutton et al. 2018)

One challenge that is found uniquely in reinforcement learning, differentiating it from supervised and unsupervised learning, is the tradeoff between exploration and exploitation. In order to obtain the maximum reward

possible, the agent has to chose the actions with the best reward possible (**exploitation**), but to discover these action, the agent has to try all actions (including actions he has never seen) in order to effectively find these preferable actions (**exploration**). In short, the agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future. The agent cannot exclusively exploit or exclusively explore without failing the reward maximization task, It has to strike a balance between the two approaches. The exploration-exploitation dilemma has been intensively studied by mathematicians for many decades, yet remains unresolved. (Richard S Sutton et al. 2018)

2.3 Reinforcement Learning Basics

2.3.1 Core Components

- **Agent and Environment**

  The agent is the decision-maker in the reinforcement learning (RL) framework, with the sole goal of maximizing the reward it receives. Everything outside it constitutes the environment. The environment not only transitions between states but also provides feedback in the form of rewards and the new state. Therefore, the interaction forms a feedback loop: after each action, the environment responds by transitioning to a new state and providing a reward R_t, which the agent uses to evaluate its performance. This interaction can be formally expressed as follows:

  $$s_{t+1} = T(s_t, a_t)$$

  where s_t is the state at time t, a_t is the action taken at time t, and T is the transition function. The reward received at time t is given by:

  $$r_t = R(s_t, a_t)$$

  The agent's goal is to maximize the cumulative reward over time (Richard S Sutton et al. 2018).

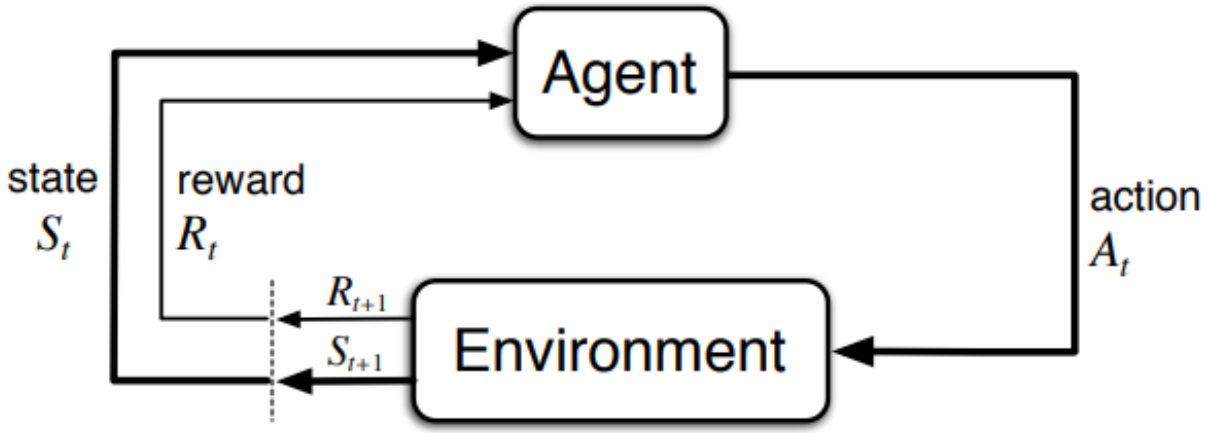


Figure 2.1: The interaction feedback loop between the agent and the environment

- **Actions (Discrete vs. Continuous)**

  Actions are central to the reinforcement learning problem, where the agent aims to find the optimal sequence of actions that maximizes the reward. Depending on the problem, actions can be either

discrete or **continuous**. The action space impacts the complexity of both the state space and the learning algorithms used. Discrete actions are simpler to handle, while continuous actions often require specialized techniques.

- **Discrete actions**: A finite set of actions, such as an agent moving in a grid with discrete movement options (e.g., left, right, up, down). The action space is then defined as a finite set:

$$A = \{a_1, a_2, \ldots, a_n\}$$

- **Continuous actions**: A continuous range of values, such as a robot rotating its mechanical arm within a specific angle range:

$$A = [0, 2\pi]$$

The choice of action space significantly impacts the complexity of the learning algorithm and the design of the agent (Richard S Sutton et al. 2018).

- **Rewards (Immediate vs. Delayed)**

In reinforcement learning, the objective is to maximize the *reward* received by the agent. This reward is a numerical signal provided by the environment at each time step. Rewards can be **immediate** or **delayed**. Immediate rewards are received right after an action is taken, while delayed rewards are obtained after a series of actions, making it necessary for the agent to plan ahead. Immediate rewards are given as:

$$r_t = R(s_t, a_t)$$

whereas delayed rewards depend on the sequence of actions and future states. The cumulative reward G_t at time t is given by:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$$

where γ is the discount factor that determines the importance of future rewards (Richard S Sutton et al. 2018).

2.3.2 Key Concepts

- **Value Function: Expected Future Rewards**

Value functions are essential in reinforcement learning as they help the agent evaluate how good a particular state (or state-action pair) is in terms of expected future rewards. The value function estimates the expected cumulative reward from a given state, guiding the agent's behavior. There are two common types of value functions:

- The **state value function** $v(s)$, which estimates the expected cumulative reward from state s:

$$v(s) = \mathbb{E}[r_t | s_t = s]$$

- The **state-action value function** $q(s, a)$, which estimates the expected cumulative reward for taking action a in state s:

$$q(s, a) = \mathbb{E}[r_t | s_t = s, a_t = a]$$

These functions are central to many RL algorithms like Q-learning and Value Iteration, which aim to find the optimal policy by evaluating these value functions (Richard S Sutton et al. 2018).

- **Policy: Mapping States to Actions**

  The agent in the reinforcement learning framework behaves according to a policy. The policy defines which action the agent should take in a given state. A policy can be deterministic or stochastic. In a *deterministic* policy, the agent always takes the same action in a given state. In a *stochastic* policy, there is a probability distribution over actions given a state. Formally, the policy π is a mapping from states to a probability distribution over possible actions:

  $$\pi(a|s) = P(a_t = a|s_t = s)$$

  The policy directly influences the agent's behavior and the outcome of the learning process (Richard S Sutton et al. 2018).

- **Model of the Environment**

  In reinforcement learning, a model of the environment is a mathematical model that predicts the transition between states following an agent's action and also predicts the reward for each state-action pair. In **model-based** RL algorithms, the agent tries to learn this model of the environment's dynamics, including state transitions and rewards. On the other hand, **model-free** algorithms, such as Q-learning, do not explicitly model the environment but instead learn an optimal policy directly from interactions. The transition model P is the probability distribution over the next state given the current state and action:

  $$P(s'|s, a) = P(s_{t+1} = s'|s_t = s, a_t = a)$$

  The reward model R predicts the reward for a given state-action pair:

  $$R(s, a) = \mathbb{E}[r_t|s_t = s, a_t = a]$$

2.4 Types of Reinforcement Learning Algorithms

Reinforcement learning (RL) algorithms can be categorized in multiple categories based on how they interact with the environment and also how they represent their decision-making process. This section outlines three major classifications: **model-based vs. model-free**, **value-based vs. policy-based**, and **on-policy vs. off-policy** methods.

2.4.1 Model-Based vs. Model-Free

- **Model-Based: Learns Environment Dynamics**

  In Model-based algorithms, the aim is to create a model of the environment as previously defined that predicts state transitions and rewards, which means that they can be used to mimic or simulate experience, allowing the agent to plan around these outcomes and improve its policy without actually interacting with the actual environment. Examples include planning algorithms in the dynamic programming paradigm and methods such as Dyna-Q. (Richard S Sutton et al. 2018).

- **Model-Free: Learns Directly from Experience**

  One limitation of model-based methods is that in most real-life situations, constructing an accurate model of the environment is extremely challenging, if not impossible. This accuracy is crucial because models need to be reasonably precise to be effective, and any inaccuracies can significantly hinder the performance of these algorithms. Model-free algorithms address this issue by bypassing the need for an explicit model of the environment. Instead, they focus on learning optimal policies or value functions directly from experience. These methods are highly popular due to their simplicity and robustness in

complex and unpredictable environments. Examples include Q-learning and Deep Q-Networks (DQN). (Richard S Sutton et al. 2018).

2.4.2 Value-Based vs. Policy-Based

- **Value-Based: Learns Value Functions**

  Value-based methods focus on estimating the value function, which represents the expected reward of states or state-action pairs, that is because policies then can be trivially derived by simply selecting actions that maximize the value function in a given state. Q-learning is a prominent value-based algorithm.

- **Policy-Based: Directly Learns Policy**

  Policy-based methods, on the other hand, directly optimize their policy, often using gradient optimization techniques. These methods are particularly useful in continuous action spaces with infinite number of actions and can be more stable in certain applications. (Richard S Sutton et al. 2018)

- **Actor-Critic: Combines Both Approaches**

  Actor-critic methods are a combination of both value-based approach and policy-based approach by estimating a policy function (actor) and a value function (critic) at the same time. The actor decides which action to choose, while the critic evaluates them, leading to faster and more stable learning (Mnih et al. 2016).

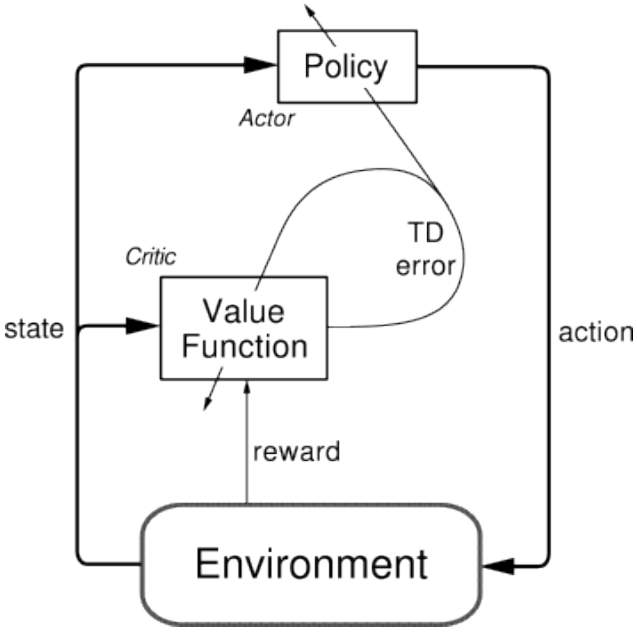


Figure 2.2: Actor-critic methods

2.4.3 On-Policy vs. Off-Policy

- **On-Policy: Learns from Current Policy**

On-policy algorithms update their policy using actions taken while following said policy. These methods, such as SARSA, aim to improve the policy currently being used, which might limit their exploration. (Richard S Sutton et al. 2018).

- **Off-Policy: Learns from a Different Policy**

  Off-policy algorithms, such as Q-learning, learn an optimal policy independently of which policy used to decide the actions being taken (called the behavior policy). This enables more flexibility and allows learning from previously collected experiences. (Watkins et al. 1992).

2.5 Q-Learning: A Fundamental RL Algorithm

Q-Learning is one of the oldest and most impactful RL algorithms in the literature. It's a **model-free off-policy** reinforcement learning algorithm that learns the optimal policy by estimating the optimal state-action value function, known as the **Q-function** $Q(s, a)$. It is based on the principles of **dynamic programming paradigm** and uses **temporal difference** learning to update the Q-values. This section will cover core concepts necessary for the Q-learning algorithm.

2.5.1 Bellman Equation

The Bellman equation serves not only as the mathematical foundation for Q-learning but also for a vast array of reinforcement learning algorithms . It defines the relationship between the value of a state-action pair and the expected rewards of taking an action at that state and following the optimal policy thereafter. Formally, the *Bellman optimality equation* for the Q-function is given by:

$$Q^*(s, a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^*(s', a')\big],$$

where Q^* is the current estimation of the Q-function, s and a are the current state and action, r is the immediate reward, s' is the next state, a' is the next action, γ is the discount factor (will be discussed in 2.5.4), and the expectation is over all possible (r, s') pairs. (Richard S Sutton et al. 2018) This recursive formula allows the Q-learning algorithm to iteratively improve its $Q^*(s, a)$. estimates.

2.5.2 The Q-Learning Algorithm

Q-learning operates by iteratively updating Q-values based on the previous equation. The update rule is defined as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big],$$

where α is the learning rate (more on that in 2.5.4). One can easily notice that this algorithm does not require a model of the environment, only previous estimates and the observations recorded from the environment. The Q-Learning algorithm is considered an *Off-Policy* algorithm because when updating its estimates it ignores the next action to be taking and only considers the actions that maximizes the Q function, allowing for the use of any behavior policy that encourages exploration (e.g. ϵ-greedy policy). (Richard S Sutton et al. 2018).

The explicit Q-Learning algorithm is detailed bellow

Algorithm 1 Q-Learning Algorithm

Input: State space S, action space A, learning rate α, discount factor γ
Output: Optimal action-value function $Q^*(s, a)$
Initialize $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A$
while *not converged* **do**
  Initialize S the start of the episode
  for *each step of the episode* **do**
    Choose action a using an exploration strategy (e.g., ϵ-greedy)
    Execute a, observe reward r and next state s'
    Q Update: $Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big]$
    Set $s \leftarrow s'$
  end
end

2.5.3 Exploration vs. Exploitation

One of the key challenges in reinforcement learning is balancing exploration and exploitation. Exploration as the name suggests involves trying new actions and possible states to gather more information about the environment, while , on the other hand, exploitation leverages the learned Q-values to choose actions that maximize the observed rewards. Strategies such as ϵ-greedy, where the agent explores with probability ϵ and exploits otherwise, are commonly used to address this trade-off. In the case of Q-learning, the learned action-value directly approximate the Q function, independent of the behavior policy which enables early convergences, and as long as you use a policy that explores enough, it is guaranteed for the Q-learning algorithm to converge. This shows the advantages of Q-learning in the **Exploration/Exploitation trade-off** (Richard S Sutton et al. 2018).

2.5.4 Learning Rate and Discount Factor

The learning rate (α) and discount factor (γ) are crucial hyper-parameters for the Q-learning algorithm:

- **Learning Rate (α)**: Determines the weight given to the new information, in our case, it is the observed difference of the current estimate from the reward plus the discounted estimate (which the both of them act as the estimate of the return) versus prior estimates. A high α leads to fast learning but may cause instability, while a low α ensures stability but slows learning.
- **Discount Factor (γ)**: an important concept of modeling the expected cumulative returns. It reflects the importance of future rewards relative to immediate rewards. A γ close to 1 emphasizes long-term rewards, while a smaller γ prioritizes immediate rewards. The smaller γ the more short sighted the agent becomes.(Richard S Sutton et al. 2018).

These parameters significantly impact the accuracy and overall performance of the Q-learning algorithm, requiring careful tuning for specific applications.

2.6 Deep Reinforcement Learning

Deep reinforcement learning (DRL) is combining classical reinforcement learning with the deep neural networks architecture to address the problem of complex decision-making tasks. DRL methods have been widely used in domains such as robotics, games-playing agents, and autonomous systems due to their ability to handle high-dimensional state and complex action spaces (Mnih et al. 2015).

2.6.1 Neural Networks in RL

Neural networks is the central piece to DRL, it is used to enhance the capabilities of traditional RL algorithms, they have multiple features and advantages which includes:

- **Function Approximation:** Neural networks can serve as function approximators to estimate value functions or policies by transforming the state features to a latent learned space usable for future estimations and calculations, enabling RL to scale to problems with large or continuous state and action spaces alike (LeCun et al. 2015).
- **Handling Complex State Spaces:** Neural networks are proving to be able to process high-dimensional and complex input spaces, such as images or sensor data, through layers of hierarchical feature extraction built on aggregating previous layers' information. (Mnih et al. 2015).
- **Feature Learning:** Neural networks are a very powerful tool for feature engineering, they learn to automatically extracting only relevant features from raw data, reducing the need for manual feature extraction, making RL applicable to a broader range of problems, where it is hard to properly define the state and action feature vectors. (LeCun et al. 2015).

Several deep learning architectures have been developed for reinforcement learning. We will present 3 of theses key architectures:

2.6.2 Deep Q-Networks (DQN):

Deep Q-Networks (DQN) (Mnih et al. 2015) combine Q-learning with deep neural networks to estimate the Q-value function. The goal is to approximate the expected reward of an action given a specific state, enabling efficient decision-making in high-dimensional state spaces. One significant limitation of DQN is its inefficiency in handling continuous action spaces. The algorithm often struggles with sample efficiency, that is because it requires a large number of interactions with the environment to effectively learn and improve.

DQN utilizes an innovative technique called **experience replay**, where past experiences are stored in a buffer and randomly sampled from during training to break correlation between consecutive data points and allows the model to retain knowledge from previous iterations.

It employs a **target network**, updated periodically, to stabilize learning and prevent drastic changes in the Q-value estimates.

DQN utilizes experience replay and a target network for stable training.

Algorithm 2 Deep Q-Network (DQN)

Input: Replay buffer D, learning rate α, discount factor γ
Output: Optimal Q-function $Q^*(s, a)$
Initialize Q-network with random weights Initialize target-network with same weights as Q-network **for** *each episode* **do**
  Initialize state s_0 **for** *each step in the episode* **do**
    Choose action a_t using ϵ-greedy policy Execute a_t, observe reward r_t and next state s_{t+1} Store (s_t, a_t, r_t, s_{t+1}) in replay buffer D Sample a random minibatch of transitions from D Compute target:

$$y = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a')$$

    Update Q-network by minimizing loss:

$$L = \mathbb{E}\big[(y - Q(s_t, a_t))^2\big]$$

    Periodically update target-network weights

DQN marked a pivotal progress in reinforcement learning, demonstrating huge performance in certain tasks. However, its reliance on handcrafted features and hyperparameter sensitivity are still a set of challenges that have to be dealt with (Hessel et al. 2018). Some Modifications such as Double DQN and Dueling DQN have been proposed to address issues like overestimation bias and suboptimal learning dynamics (Van Hasselt et al. 2016). There is also Extensions like Rainbow DQN that integrate multiple improvements to create a more robust variant of the algorithm.

2.6.3 REINFORCE (Policy Gradiant):

REINFORCE is a **Policy Gradient methods** which means that it directly optimizes the policy by maximizing the Gradient of the in order to maximize the expected cumulative reward (Richard S. Sutton et al. 1999). Unlike value-based methods, policy gradients methods do not rely on value function approximation and can handle stochastic policies naturally. A key advantage of REINFORCE is its simplicity and compatibility with neural network architectures for approximating policies, and combined with its adaptability for environments with continuous action spaces (given that it optimize the policy directly), makes it a key component in the broader reinforcement learning ecosystem and a popular choice for robotics and control problems where continuous actions are very common. However, the algorithm can suffer from inefficiency due to the high variance resulting from computing gradient estimates using **Monte Carlo estimates** of the return, requiring techniques like **baseline subtraction** to stabilize training (Richard S. Sutton et al. 1999).

Policy Gradient methods, such as REINFORCE form the foundation of more advanced algorithms like **Actor-Critic**, which address its limitations by incorporating value-based learning.

Algorithm 3 Policy Gradient (REINFORCE)

Input: Policy parameter θ, learning rate α
Output: Optimized policy $\pi_\theta(a|s)$
for *each episode* **do**
  Generate an episode by following policy $\pi_\theta(a|s)$
  for *each time step t in the episode* **do**
    Compute return $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ Update policy parameters:

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \ln \pi_\theta(a_t|s_t)$$

REINFORCE is often combined with **entropy regularization** to encourage exploration and prevent premature convergence (Williams 1992).

2.6.4 Proximal Policy Optimization (PPO):

Proximal Policy Optimization (PPO) is a state-of-the-art policy optimization algorithm classified as an actor-critic method. It was designed to address the challenges of stability and sample efficiency in reinforcement learning (Schulman et al. 2017). PPO is categorized as an on-policy algorithm that alternates between data collection (via the current policy) and gradient updates, which in turn leverages the collected data efficiently.

PPO simplifies the complex trust region optimization used in Trust Region Policy Optimization (**TRPO**) by introducing a **clipped surrogate objective**,that is designed to restrict the magnitude of policy updates, guaranteeing stable learning and prevents colossal updates (Schulman et al. 2017). The clipped objective is combined with the value loss objective and is complemented by **entropy regularization**, which adds an incentive for exploration, ensuring that the policy does not overfit and maintains sufficient diversity in the action space and avoids premature convergence (Schulman et al. 2017). The final loss function is defined as:

$$L = L^{clip}(\theta) - c_1 L^{VF}(\theta) + c_2 L^{Entropy}(\theta)$$

where:

- $L^{clip}(\theta)$ is the clipped surrogate loss that prevents large policy updates,
- $L^{VF}(\theta)$ is the loss based on the value function which evaluates how well the policy estimates the expected future rewards,
- $L^{Entropy}(\theta)$ is the entropy loss that encourages exploration,
- c_1 and c_2 are hyper-parameters controlling the weight of the value function loss and entropy bonus, respectively.

PPO's ability to balance exploration and exploitation and being able to supports both discrete and continuous action spaces, effectively contributes to its robustness and reliability, making it versatile for various tasks, including robotics, games, and control problems. These reason justify why PPO has been extensively used in solving challenging tasks, including OpenAI's robotics simulations and the multiplayer online video game *Dota 2* (Berner et al. 2019).

Algorithm 4 Proximal Policy Optimization (PPO)

Input: clipping threshold ϵ, policy learning rate α, value learning rate β
entropy coefficient c_2, value loss coefficient c_1.
Output: Optimized policy $\pi_\theta(a|s)$
Initialize policy π_θ and value function V_ϕ **for** *each iteration* **do**
  Collect trajectory using current policy π_θ $\{s_t, a_t, r_t\}_{t=1}^{T}$
  Compute the advantage estimates:

$$A(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

for *each epoch* **do**
  for *each minibatch* **do**
    Compute ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

    Clipped surrogate objective:

$$L^{clip}(\theta) = \mathbb{E}\big[\min(r_t(\theta)A(s_t, a_t), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A(s_t, a_t))\big]$$

    Value function loss:

$$L^{VF}(\theta) = (V_\phi(s_t) - (r_t + \gamma V_\phi(s_{t+1})))^2$$

    Compute Entropy measure for the current policy as the entropy loss:

$$L^{Entropy}(\theta) = -\sum_a \pi_\theta(a|s) log(\pi_\theta(a|s))$$

    Total loss:

$$L = L^{clip}(\theta) - c_1 L^{VF}(\theta) + c_2 L^{Entropy}(\theta)$$

    Update θ using stochastic gradient ascent on L:

$$\theta \leftarrow \theta + \alpha \nabla_\theta L$$

    Update ϕ using stochastic gradient descent on L^{VF}:

$$\phi \leftarrow \phi - \beta \nabla_\phi L^{VF}(\phi)$$

Conclusion

This chapter has explored the foundations and advanced applications of Reinforcement Learning, from its distinctive characteristics as a machine learning paradigm to its practical implementations. We examined core concepts like states, actions, and rewards, which form the basis for both traditional algorithms and modern approaches. The progression from basic Q-learning to sophisticated deep RL architectures demonstrates how neural networks have expanded the field's capabilities. Through various algorithmic approaches RL has evolved into a powerful framework for solving complex sequential decision-making problems. The emergence of architectures like DQN and PPO showcases RL's potential in addressing real-world challenges through the combination of classical principles with deep learning.

Chapter 3

Reinforcement Learning for Automatic Code Optimization

3.1 Reinforcement learning in Tiramsu

Tiramisu is a polyhedral compiler framework designed to generate high-performance code for multicore CPUs, GPUs, and distributed systems. It features a flexible scheduling language, explicit control over data layout and communication, and supports advanced optimizations for domains like deep learning, image processing, and linear algebra (Baghdadi et al. 2018).

Recent work has explored using reinforcement learning (RL) to optimize Tiramisu code (Hennouni et al. 2022; Lamouri et al. 2024). While both works aim to automate and improve scheduling, the key difference lies in the second work's introduction of a syntax tree representation. This structure allows for a more refined understanding of the program's hierarchical relationships, enhancing the RL agent's ability to make informed optimization decisions.

3.1.1 Hennouni and El Hassane

We start with the earlier work of Hennouni and El Hassane, the following architecture was proposed:

Environment

The environment is modeled using a Markov Decision Process (MDP) to represent the search space, as shown in Figure 3.1. States represent observations of the environment at a given time. Transitions between states are driven by actions, which correspond to specific code transformations with associated parameters. If a transformation is valid, the system transitions to a new state with updated observations; otherwise, it remains in the current state until a valid action is selected.

Observation

Tiramisu programs, in their raw form, cannot be directly observed and understood by the agent. To address this, a feature extraction process is applied to transform the programs into a compact vector representation that captures only the essential features. This representation forms the first part of the observation. The second part of the observation is a mask, a vector of the same size as the number of actions, where each element is either 1 or 0. A value of 1 at position i indicates that action i is valid, while a value of 0 means the action is invalid and cannot be chosen by the agent. This mask restricts the available actions for the agent, ensuring that each action can only be applied once per program.

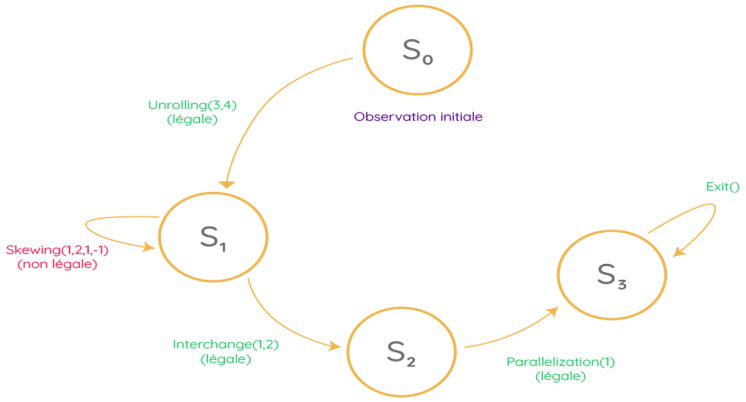


Figure 3.1: Markov decision process representing a search space exploration (Hennouni et al. 2022)

Actions

in this work, transformations are applied to Tiramisu code, including operations such as Interchange, Tiling, Skewing, Unrolling, Parallelization, and Reversal, with an "Exit" action to end the sequence. Managing the large and variable number of potential actions, which depends on the number of loops in the program, presents a significant challenge.

To address this, a strategy is proposed where the action space is simplified by first selecting a transformation and then randomly choosing its parameters. This approach streamlines decision-making by focusing solely on the transformations while allowing the parameters to vary randomly, reducing complexity and enabling a broader exploration of the potential schedules.

Agent

The agent is modeled using a single neural network that takes the observation (program representation and mask) as input. The network has two outputs: the action probabilities (policy) and the predicted final speedup (value). It uses a feed forward, fully connected architecture, with several hidden layers to capture complex patterns in the input data as shown in figure 3.2 . This design helps the agent learn effectively and make informed decisions, optimizing both the policy and value simultaneously.

Reward types and formulas

In the proposed method, two types of rewards are used:

- **Final reward**: Given at the end of the episode, based on the speedup achieved by the final schedule. This allows for faster learning but lacks immediate feedback on intermediate actions.
- **Immediate reward**: Provided at each timestep, enabling the agent to learn from every action. However, it consumes more resources and slows down learning.

To calculate these rewards, three formulas were tested:

- **Simple reward**: The most intuitive formula, but it can lead to positive rewards even for poor schedules.

$$\text{Simple reward} = \begin{cases} -1 & \text{if illegal action or error} \\ \frac{\text{Initial Execution Time}}{\text{Final Execution Time}} & \text{otherwise} \end{cases}$$

- **Relative reward**: Penalizes worse execution times, but the penalties can be unbounded.

$$\text{Relative reward} = \begin{cases} -1 & \text{if illegal action or error} \\ \frac{\text{Final Execution Time} - \text{Initial Execution Time}}{\text{Final Execution Time}} & \text{otherwise} \end{cases}$$

- **Non-relative reward**: Balances the rewards and penalties, encouraging the agent to improve speedup without being influenced by illegal actions or errors.

$$\text{Non-relative reward} = \begin{cases} 0 & \text{if illegal action or error} \\ \frac{\text{Initial Execution Time}}{\text{Final Execution Time}} & \text{if speedup} \geq 1 \\ -\frac{\text{Final Execution Time}}{\text{Initial Execution Time}} & \text{if speedup} < 1 \end{cases}$$

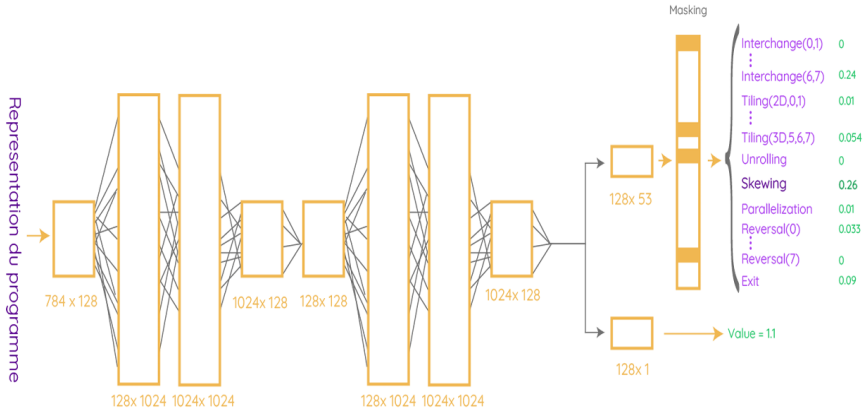


Figure 3.2: Architecture of the model proposed in (Hennouni et al. 2022)

3.1.2 Lamouri and Merad

in this work (Lamouri et al. 2024), a different architecture was proposed that utilizes an encoded feature vector derived from the Abstract Syntax Tree (AST) representation of a for loop. The following outlines the key components of their solution:

State

The key difference between the two works lies in how the state is represented. In this work, the state is encoded by passing the Abstract Syntax Tree (AST) representation of a loop through an encoding network, which outputs a feature vector. However, this approach does not capture all the information about the loops, resulting in an incomplete representation of the state. Consequently, the system transitions from being a fully observed Markov Decision Process (MDP) to a Partially Observed Markov Decision Process (POMDP), as the state no longer satisfies the Markov property due to missing information.

In a POMDP, the state is partially observed, as is often the case in real-world scenarios where complete information about the system is unavailable. This incomplete observation requires additional techniques to stabilize learning and achieve optimal results, as the Markov property no longer holds.

Actions

To avoid designing a policy network π with an output containing an excessively large number of actions resulting from all possible parameter combinations, the authors restricted parameters with negligible impact on performance. For example, recognizing that parallelization is more effective on outer loops, they excluded parallelization for inner loops. This approach resulted in the definition of the following 27 actions:

- **Parallelization**: At loop levels 0 and 1.
- **Skewing**: Between loop levels (0,1) and (1,2).
- **Unrolling**: With factors $4, 8, 16$.
- **2D Tiling**: At levels (0,1), (1,2), (2,3), and (3,4) with factors $x = 32$ and $y = 32$.
- **3D Tiling**: At levels (0,1,2), (1,2,3), and (2,3,4) with factors $x = 32$, $y = 32$, and $z = 32$.
- **Inversion**: Of loops from 0 to 4.
- **Permutation**: Of loops (0,1), (1,2), (2,3), (3,4), (4,5), and (0,2), (1,3), (2,4).
- **Next**: To switch the branch or terminate the episode.

Policy

The policy, denoted by π, is a function that maps states to actions and defines the probability of choosing an action in a given state. In a standard Markov Decision Process (MDP), the state is fully observed, but in our case, the state is partially observed using an encoder, which leads to a loss of information. This makes the problem a Partially Observed Markov Decision Process (POMDP), where decisions are based on observations, not the full state. To make better decisions, the agent needs to consider the history of actions and observations, which is known as belief state construction.

To handle such problems, Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, are used to process sequential data. LSTMs encode the history of observations and actions into hidden vectors, allowing the agent to make decisions based on the past. The LSTM policy π_{LSTM} takes observations, hidden states, and memory vectors to determine the probability of selecting an action as shown in figure 3.3.

The architecture of the LSTM policy includes two parts: "The Critic," which evaluates the value of a state, and "The Actor," which selects actions based on the policy. The Critic helps to correct bad decisions, while the Actor uses a special mask to avoid undesirable actions, such as repeating an action. This method helps the agent choose the most appropriate action, particularly in complex environments.

In this case, the agent utilizes PPO, along with the LSTM-based policy, to make decisions that are informed by its past experiences while ensuring stable and effective performance.

Rewards

The reward used in this context is the logarithm of the acceleration rate (τ_i) achieved by an optimization action, defined as $r_i = \log(\tau_i)$. This reward structure ensures that actions slowing down execution ($\tau_i < 1$) result in negative rewards, actions with no effect ($\tau_i = 1$) yield zero rewards, and actions accelerating execution ($\tau_i > 1$) produce positive rewards. The logarithmic transformation converts the product of acceleration rates into a summable form, aligns with the reinforcement learning objective of maximizing cumulative rewards, penalizes detrimental actions, and reduces variance in reward values, thereby stabilizing the learning process.

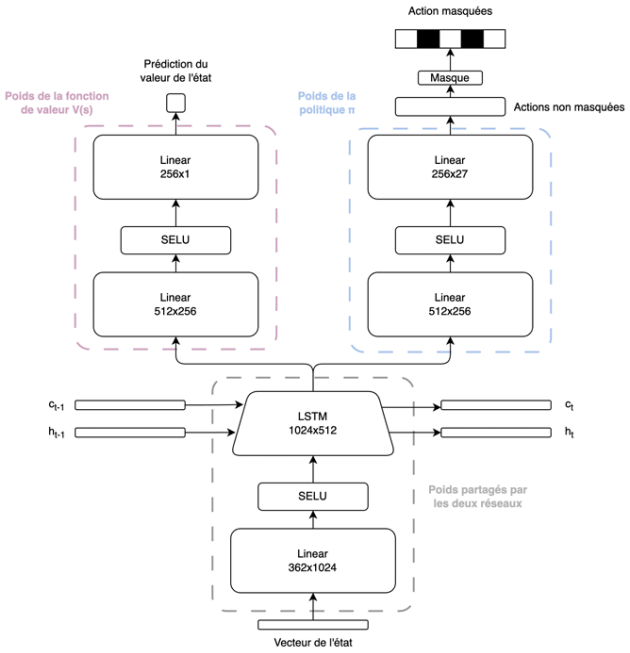


Figure 3.3: Structure of the policy network

3.2 CompilerGym

The field of compiler optimization has seen significant advancements with the introduction of artificial intelligence techniques like reinforcement learning (RL). These approaches aim to replace or augment traditional handcrafted heuristics by leveraging empirical data to make optimization decisions. Despite RL's success in outperforming human experts, progress in this domain is often limited by the experimental infrastructure's complexity. CompilerGym, introduced by Cummins et al. (2022) (Cummins et al. 2021), addresses these challenges by providing a robust suite of environments tailored to real-world compiler optimization tasks.

CompilerGym's design supports scalable, efficient, and extensible experimentation, offering an architecture that combines flexibility with high performance. It is particularly suited for tasks such as optimizing code size, runtime, and GPU workloads, with performance improvements that make it a game-changer in compiler research.

3.2.1 System Architecture

CompilerGym's architecture is divided into two main components:

Frontend

The frontend is a Python library that exposes compiler optimization tasks through environments designed to integrate seamlessly with RL workflows. It leverages the OpenAI Gym framework, making it familiar and accessible to researchers working on RL. Figure 3.4 It illustrates the structure of RL environments within CompilerGym.

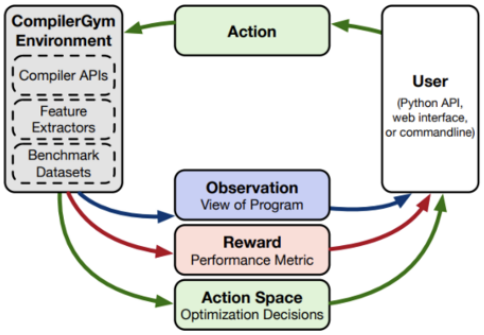


Figure 3.4: Compilergym frontend

Backend

The backend employs a client-server architecture, enabling extensibility and scalability. It supports environments for three specific compiler optimization problems and is designed for experimentation at scale. The backend outperforms prior works by being 27 times faster, offering broader search spaces, and supporting millions of benchmarks for training RL agents. Figure 3.5 showcases the backend's functionality, which currently includes the following environments:

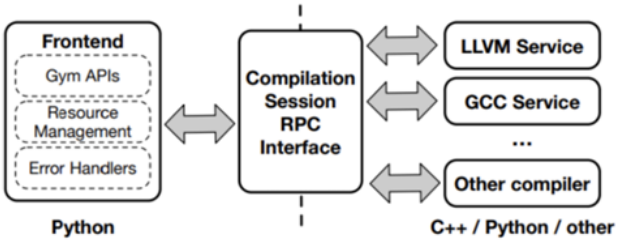


Figure 3.5: CompilerGym backend

LLVM Phase Ordering

LLVM (Low-Level Virtual Machine) is a widely used modular compiler infrastructure that processes input source programs into an intermediate representation (IR). This representation is language-agnostic and undergoes optimization through a configurable pipeline of passes. The sequence of these optimization passes, known as the "phase order," significantly impacts the final binary's quality.

- **Action Space:** The action space consists of a discrete selection of 124 optimization passes automatically extracted from LLVM. These passes represent various optimization techniques that can be applied to the intermediate representation (IR) of a program. RL agents can select from these passes in a sequence, with the objective of optimizing the program's performance, code size, or binary size.
- **Reward Metrics:** The reward metrics include code size, binary size, and runtime performance. Code size refers to the number of instructions in the source code, binary size measures the size of the compiled object file, and runtime performance evaluates the execution time on specific hardware.

- **Observations:** There are five different observation spaces to describe the state of the environment. These observations capture features such as the complexity of the program, the applied optimization passes, the performance metrics, and the configuration of the compiler.

GCC Flag Selection

This environment exposes the optimization space defined by the GCC compiler's command-line flags and is compatible with all GCC versions.

- **Action Space:** The action space for GCC flag selection offers two representations:
  - A list of integers that encode different optimization flag choices.
  - A flattened categorical list, optimized for RL tools.

  Both representations allow RL agents to select different combinations of flags for optimization.
- **Reward Metrics:** Reward metrics include assembly size and object code size. Assembly size measures the size of the generated assembly code, while object code size measures the size of the compiled object file.
- **Observations:** There are four observation spaces that provide information about the compilation process. These include features of the source code, the history of optimization flags used, the runtime performance of the compiled program, and the internal state of the compiler.

CUDA Loop Nest Generation

CompilerGym extends its functionality to GPU workloads through a specialized environment for tuning CUDA loop nests. It integrates `loop_tool`, a minimalist compiler for dense linear algebra, which decomposes BLAS-like routines into directed acyclic graphs (DAGs) of n-dimensional primitive arithmetic operations.

- **Action Space:** The action space in CUDA loop nest generation consists of discrete actions that modify the loop structures in the CUDA code. This includes interactions such as loop reordering, tiling, unrolling, and adjusting block sizes to optimize performance on the GPU.
- **Reward Metrics:** The reward is based on the floating-point operations per second (FLOPs), which measure the computational efficiency of the optimized code. Higher FLOPs indicate better performance.
- **Observations:** The environment provides two observation spaces: one describing the structure of the loop nests, including the number of loops and their dependencies, and another providing performance profiling data, such as execution time and memory usage.

By standardizing experimentation and providing a high-performance framework, CompilerGym accelerates the exploration of RL-based solutions in compiler research. Its design facilitates integrating cutting-edge techniques into production compilers while ensuring reproducibility and scalability.

3.3 Halide

Halide (Ragan-Kelley et al. 2013), is a *domain-specific language* (DSL) for computer vision and especially for the development of image processing programs. Its main selling point is its ability to decouple the algorithms' description and definition from their schedules, in other words the *how* to compute is separated from *what*

to compute. This separation allows programmers to optimize performance by independently specifying how computation is mapped to hardware-level directives without altering the high-level algorithm, while keeping the code clean, understandable and achieving satisfactory code performance without any programmer pain. Nonetheless, the problem of finding the best possible schedule still persists, researchers have explored various techniques to automate the process of optimizing Halide schedules, we will explore two interesting approaches that has been proposed in the literature.

Autotuning Using Genetic Search

The initial approach to Halide schedule optimization involved autotuning through a genetic algorithm, as proposed by Ragan-Kelley et al. in their seminal work on Halide (Ragan-Kelley et al. 2013). By leveraging the evolutionary process of the genetic algorithm, it explores the vast search space of possible schedules to find an approximate solution. Genetic algorithms work by generating an initial population of candidate solution (in our case: schedules), evaluates their performance, and iteratively improves the population through selection, crossover and mutation.

The autotuning process begins with a randomly initialized population of schedules. Each candidate schedule is evaluated based on its execution runtime performance. The best-performing schedules are selected as "parents" to produce the next generation (elitist approach), selecting two parents using *tournament selection* and crossing them by the *two-point crossover* method and finally mutating the "children" by chosing a function at random and randomly change its schedule. Over successive generations, the algorithm will hopefully converges towards high-performant schedule.

This method was crucial in demonstrating Halide's potential to achieve performance equal to or superior to hand-tuned implementations across various platforms. Ragan-Kelley et al. showed that genetic search could efficiently identify schedules that efficiently utilize hardware resources, such as parallelization, vectorization, and locality based cache management. However, for complex pipelines and programs, this method needs incorporating prior knowledge to the optimization task otherwise it will struggle to overcome local minima and to converge rapidly.

Halide Schedule Optimization with Reinforcement Learning

To address the limitations of the first approach, Pecenin et al.(Pecenin et al. 2019) proposed a novel approach using a reinforcement learning agent as a control and optimization heuristic for Halide programs. They chose the PPO algorithm for the agent's development for its simplicity and the fact that it supports high dimensional actions scenarios, with continuous action space or large discrete actions space.

The RL agent is given as input an Image Processing Pipeline (i.e. the halide program to be programmed) and some scheduling options from the user based on his domain knowledge which will help in accelerating the process because it reduces the search space. The agent interacts with the environment through a series of candidate schedules that the environment will execute and measure the runtime and gives from there the reward to the agent to learn from.

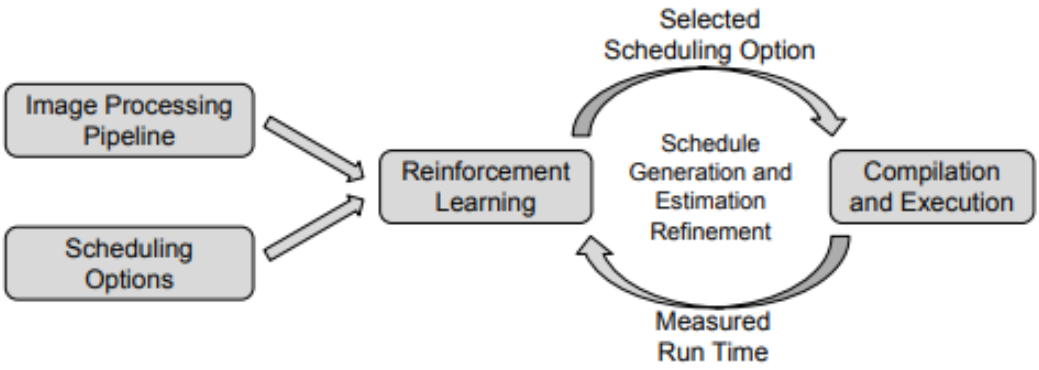


Figure 3.6: Halide Reinforcement Learning Environment (Pecenin et al. 2019)

- **The State:** The state in the Halide RL environment represents the current Halide schedule configuration which includes the currently applied optimizations such as loop order, tiling sizes, vectorization, parallelism and their order. It is expressed as the set of chosen halide directives that are a part of the scheduling options. Each scheduling option is expressed as a tuple $< s, d, A >$ of possible S stages (the functions or pipelines in the halide program) , D Directives (possible code transformations) and A the set of their parameters. Every option is given a unique numerical identifier, with which the state gets encoded as a feature vector providing a compact representation of the schedule, that is then used as an input to the policy and value networks in the PPO algorithm. In the figure 3.7 is excerpt of the search space generated by the set of scheduling options.

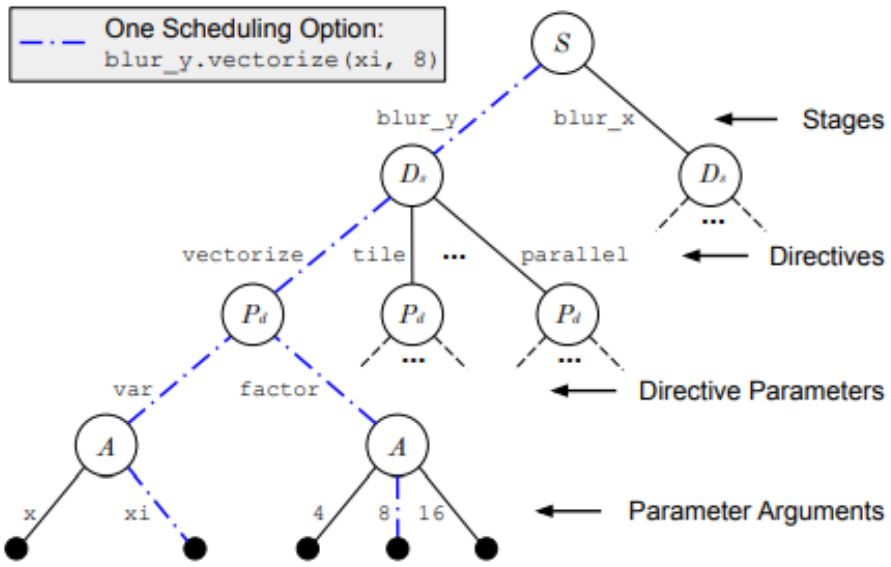


Figure 3.7: Halide Search Space expressed a arborescent structure

- **The Actions:** In the Halide RL framework, an action is applying one scheduling option $< s, d, A >$

to the current execution schedule. It include as discussed previously applying a directive d be it vectorization, parallelization and unrolling ...etc to a stage s, which represents any function in the program, with a list of parameters A. They also include a special operation *no operation* that finalizes the schedules.

- **The Reward:** The reward is a scalar value the agent receive after choosing an action from the environment. The PPO agent receive a positive value if the current action led to a decrease in the current runtime rt_{curr} compared to the previous one rt_{prev}, and it will only get a negative value -1 when the chosen action leads to an error. The reward system that was chosen, does not penalize the chose that increases the current runtime, in hope of finding a scheduling option that is more rewarding in the long run (future reward). the rewards are normalized by dividing by the factor $\frac{rt_{init}}{100}$, so that different programs can have the same reward for similar schedules performance wise.

$$r = \begin{cases} \frac{(rt_{prev} - rt_{curr}) \times 100}{rt_{init}} & \textbf{if} \quad rt_{curr} < rt_{prev} \\ 0 & \textbf{if} \quad rt_{curr} >= rt_{prev} \\ \text{-1} & \textbf{if} \quad error \end{cases}$$

3.4 PolyGym

The polyhedral model provides a structured way to define semantics-preserving transformations that can be used to improve the performance of a wide range of loops. Finding profitable transformations in this space is a difficult problem that is usually solved using domain knowledge-based heuristics and depends on specific loop shapes. PolyGym (Brauckmann et al. 2021) came to overcome this constraint through its representation of the problem as a Markov Decision Process (MDP). This allowed for an agnostic formulation of the legal transformation space in the polyhedral model. Using this generic MDP formulation, reinforcement learning can be used to learn optimization policies across a wide range of loops.

3.4.1 Polyhedral Compilation

The polyhedral model is a compilation model based on an algebraic representation of programs involving loop nests. It's used to describe program behavior in terms of sets of points that represent program iterations and form an algebraic structure called a lattice polyhedron in a multidimensional space.

Polyhedral compilers can perform complex restructuring of loop nests on static control, regular loop nests, and general program parts with data-dependent control flow. Polyhedral techniques can work at the granularity of their elements, meaning at the granularity of a loop iteration and instruction instance. This allows the polyhedral model to construct complex sequences of loop transformations.

It consists of a set of constraints and parameters. The constraints are a set of linear inequalities and equalities that define polyhedra. In PolyGym, a schedule represents semantics-preserving transformations. A schedule is defined as a vector (a transformation function) in the case of an affine function. Otherwise, the transformation can be expressed in multidimensional schedules using Farkas' lemma, which allows transforming affine functions respecting these dependencies into a set of inequalities.

3.4.2 The Markov Decision Process of PolyGym

A schedule represents the execution order of instructions within a loop. For example, if we have a "for" loop containing the instruction $y[i] = 0$, and the number of iterations $N = 5$, then the schedule can be represented as an array: $S = [0, 1, 2, 3, 4]$, where $S[i]$ represents the execution order of iteration i. Any permutation of this order that preserves the dependencies of the different program instructions will produce the same result as the original code. For example, it is also possible to execute the "for" loop instructions in this order: $S = [2, 0, 3, 1, 4]$, as it maintains program dependencies and produces the same result.

Optimizing a loop means finding a good schedule and target architecture. The construction of the schedule space is done in two distinct steps: The construction of a valid schedule space, and its exploration to find a profitable schedule. Thus, two MDPs are created to solve these two problems. Even though the state space of the second MDP depends on the options chosen in the first, these are two distinct MDP formulations. Consequently, there are two possibilities for exploring the environment: either a single combined RL agent operates on the combined space, or two distinct agents explore each of the two MDPs.

3.4.3 Schedule Spaces

In the first MDP illustrated in figure 3.8 , valid schedules are represented as multidimensional schedules to allow exploitation of a powerful result from discrete geometry called Farkas' lemma. This multidimensional representation is defined as:

$$S_{\text{cons}} = \{(i_{\text{dim}}, i_{\text{dep}}, d_1, \ldots, d_{|D|}) \mid i_{\text{dim}}, i_{\text{dep}}, d_1, \ldots, d_{|d|} \in \mathbb{N}, i_{\text{dim}} > 0\}$$

where i_{dim} represents the current dimension of the schedule, the second component i_{dep} represents the current instruction dependency being selected, while the other components represent strong dependencies included in this dimension. In Farkas' lemma, strong dependencies are strictly greater or lesser inequalities. The action set is defined as $Act_{\text{cons}} = \{\text{next\_dim}, \text{next\_dep}, \text{select\_dep}\}$, where next_dim increases the dimension and select_dep adds the current dependency to the set of strong dependencies if it hasn't been added previously. The next_dep action increases the current available dependency, skipping those that have already been selected.

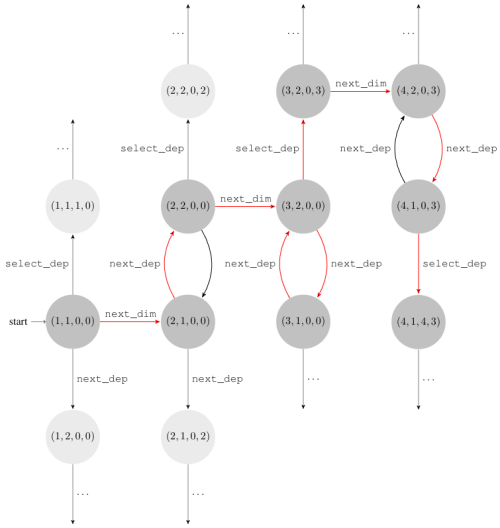


Figure 3.8: construction of the first MDP

3.4.4 Coefficient Space

Once the schedule space has been generated, profitable schedules follow this rule: If an arbitrary point p of the polytope, representing the set of feasible solutions of inequalities corresponding to instruction dependencies (result of Farkas' lemma), can be written as a convex combination of vertices v_i (i.e., $\lambda_i \geq 0$ for all $i = 1, \ldots, s$, and $\sum \lambda_i = 1$), and a positive linear combination of rays r_i (i.e., $\alpha_i \geq 0$ for all $i = 1, \ldots, t$), then it lies within the polytope. This can be represented as:

$$p = \sum_{i=1}^{s} \lambda_i v_i + \sum_{i=1}^{t} \alpha_i r_i$$

The specific state space for this second MDP illustrated in figure 3.9 is defined as:

$$S_{\text{expl}} = \{(\lambda_{1,1}, \ldots, \lambda_{1,s_1}, \alpha_{1,1}, \ldots, \alpha_{1,t_1}, \ldots, \lambda_{k,1}, \ldots, \alpha_{k,t_k}) \mid \lambda_{i,j}, \alpha_{i,j} \in \{0, \ldots, N\} \text{ for all } i, j\}$$

The agent uses the `select_coeff` function (representing an action) to find the values of coefficients $i = 1, \ldots, s$ and $i = 1, \ldots, t$. This is accomplished by iterating over all vertices and rays and selecting a coefficient each time.

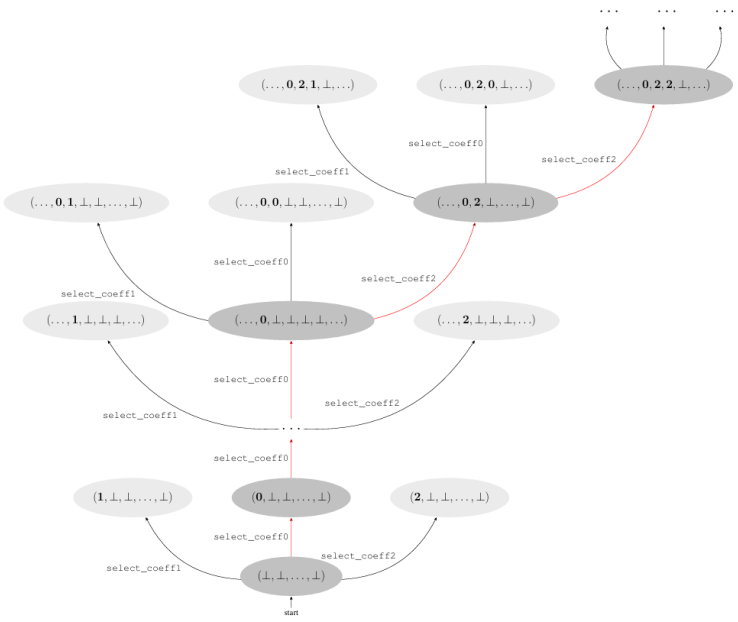


Figure 3.9: Exploration of the coefficient space in the second MDP

3.4.5 Rewards

A feedback loop is defined by the rewards of an MDP or environment. To determine the reward, the two separate MDPs produce a single schedule, which is then compiled with the LLVM-Polly tool and executed. Based on this final schedule, rewards for both individual MDPs are defined uniformly, where each MDP receives a positive reward if this final schedule is valid. A zero reward is returned if the action results in an incomplete program, and a negative reward is returned if it leads to an invalid state.

3.5 A Reinforcement Learning Environment for Automatic Code Optimization in the MLIR Compiler

This work (Bendib et al. 2024) introduces a novel reinforcement learning (RL) environment tailored for the MLIR compiler, aimed at automating code optimization. By integrating RL techniques into the MLIR frame-

work, the study addresses the need for more flexible and efficient optimization strategies in modern compilers.

The environment leverages a hierarchical action space, breaking down the optimization process into smaller subspaces to enable efficient exploration. The proposed Multi-Action RL agent selects both the types of transformations and their parameters, offering greater optimization flexibility compared to traditional methods.

The following represents the main components of the RL system used in their solution:

- **State:** Using textual MLIR code as input to the agent is impractical; therefore, a feature extraction step, as illustrated in Figure 3.10, is performed to construct a feature vector. The state representation is a compact feature vector derived from key properties of the operation being optimized. These properties include loop bounds, load and store access matrices, the count of mathematical operations, and a history of previously applied optimizations. For example, access matrices capture memory patterns by describing how data is loaded and stored within a loop nest. A padding mechanism ensures consistent feature sizes across varying loop and operation dimensions. This representation provides the RL agent with a comprehensive understanding of the optimization landscape.

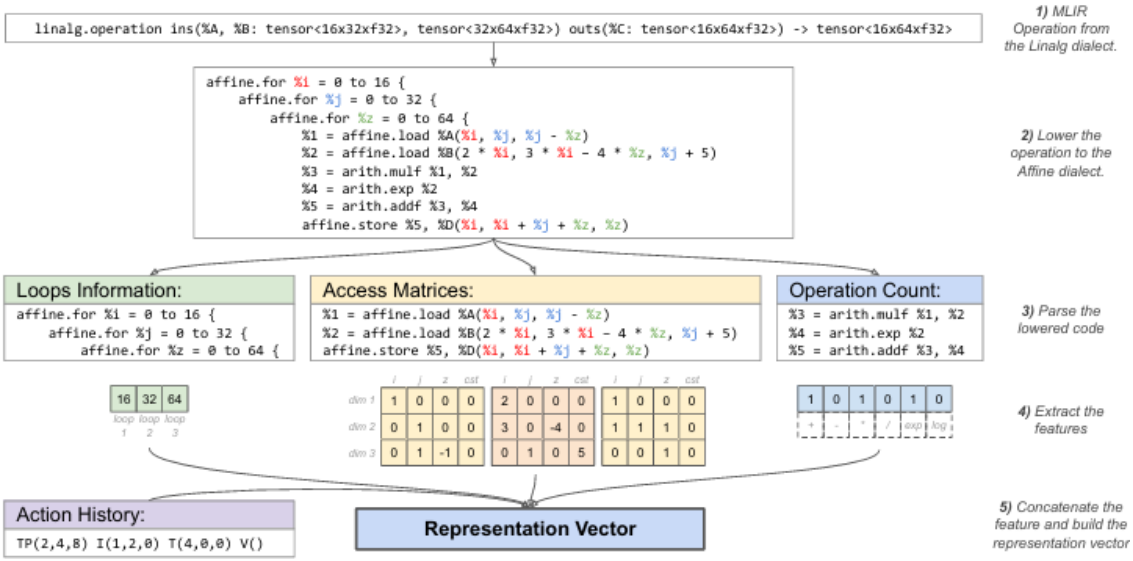


Figure 3.10: Example of a feature extraction for an MLIR operation

- **Actions:** Actions are transformations applied to operations, including tiling, parallelization, interchange, vectorization, and Im2col (Image-to-Column an operation popular in deep neural network that facilitates convolution operation). Each action has specific parameters, such as tile sizes or loop permutations. For instance, tiling divides loops into smaller sub-loops with a chosen tile size, while interchange rearranges loop orders to optimize memory access. The hierarchical action space structure, shown in Figure 3.11, represents each action as a tuple of sub-actions. The agent begins by selecting the type of transformation and subsequently determines the associated parameters.

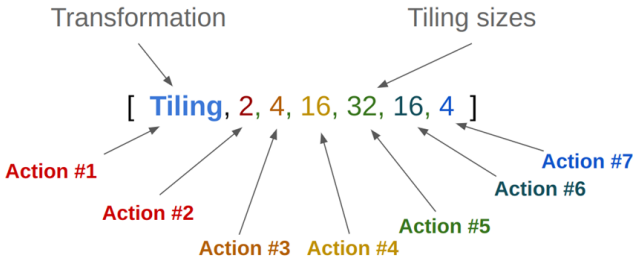


Figure 3.11: Representation of Tiling operation in the hierarchical action space

- **Reward:** The reward function evaluates the performance improvement achieved by the applied transformations, primarily using the speedup of execution time. Two reward setups are proposed: immediate rewards provide feedback after each action, whereas final rewards are computed only at the end of the optimization sequence. The logarithm of the speedup is used to facilitate reward accumulation. Additionally, an adaptive timeout mechanism penalizes excessively long execution times, ensuring stable training and effective optimization across diverse operations

3.5.1 Policy:

The policy is what defines the behavior of the agent inside the environment, in deep RL the policy is represented using a neural network. The policy is divided into 3 parts: the backbone, the value network and the policy network. In the following we will detail each part:

- **Backbone:** The backbone comprises four dense layers, each with 512 nodes and ReLU activation functions between them. It produces a feature vector that serves as input to both the value network and the policy network.
- **Value Network:** In the context of Proximal Policy Optimization (PPO), this is referred to as the critic. It predicts the expected reward and plays a crucial role in guiding the agent's policy. The network consists of four dense layers each with 512 nodes with Relu activation in between, and a final single node layer to output the reward.
- **Policy Network:** The action space is divided into multiple sub-actions, requiring the network to select several actions to construct the final one. Figure 3.12 illustrates the structure of the policy network.

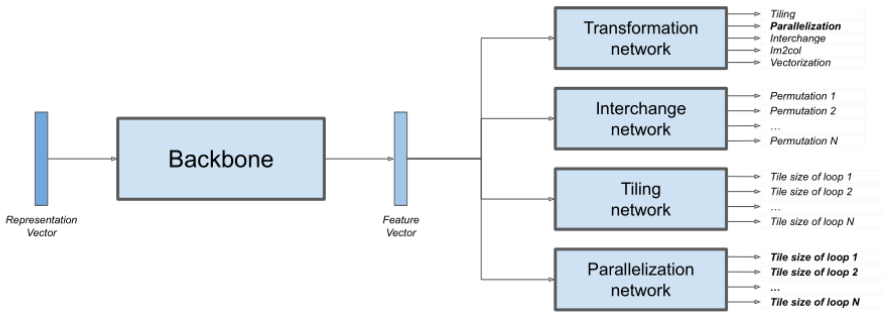


Figure 3.12: Policy network structure

The process begins by passing the feature vector through the transformation network to determine the transformation to apply. If the selected transformation requires parameters (e.g., Interchange, Tiling, Parallelization), the feature vector is then passed to the corresponding parameter-specific network:

- **Tiling and Parallelization:** Each loop can have $M+1$ possible tile sizes including 0 for no tiling. The model employs two dense layers: the first with 512 nodes and the second with $N \times (M+1)$ nodes as shown in figure 3.13. The output is reshaped into $(N, M+1)$, representing the distribution over the $M+1$ tile sizes for each of the N loops. The tile size for each loop is then selected based on this distribution.
- **Interchange:** We determine the permutation to apply from N possible consecutive permutations. This is achieved using two dense layers: the first with 512 nodes and the second with N nodes. A distribution is generated over these N nodes, from which an action is sampled to represent the selected permutation.
- **Vectorization and im2col:** Vectorization and im2col don't require any parameters therefore do not need require a corresponding network.

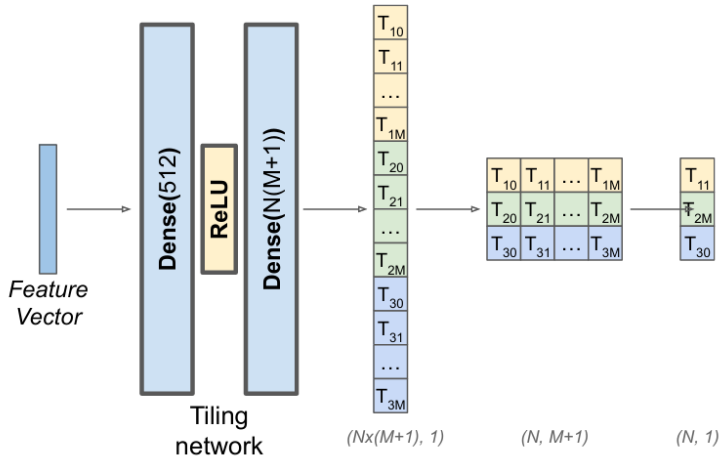


Figure 3.13: Tiling network architecture

3.6 Synthesis

The table 3.1 below presents a comparative analysis of the various works illustrated throughout this chapter. These solutions differ in their design choices, modeling techniques, and target objectives, providing a diverse landscape of approaches to solving optimization challenges in compilers. The comparison focuses on several key aspects, including the use of the polyhedral model, cost model integration, role in the RL workflow, and state representation.

Analysis

The table highlights the distinct features and methodologies of each framework:

- **Use of the Polyhedral Model:** *Tiramisu* and *PolyGym* are the only frameworks leveraging the polyhedral model, which excels in optimizing loop-based computations. This choice suggests a focus on

	Tiramisu	**Halide**	**PolyGym**	**CompilerGym**	**MLIR-RL**
Use polyhedral model	YES	NO	YES	NO	NO
Use cost model	YES	NO	NO	NO	NO
Use in RL solution	Agent	Agent	Environment	Environment	Agent
Used algorithm	PPO	PPO	No agent to train	No agent to train	PPO
Modeling using MDP	YES	YES	YES	YES	YES
Number of Actions	53	Depend on the task	Depend on the use case	Depend on the environment	Depend on the transformation chosen
State representation	Observation vector of code features	Halide directives and their parameters	Point in the graph	Intermediate Representation (IR) of the code	Observation vector of code features
Reward	Speedup	Speedup	Speedup	Speedup	Speedup

Table 3.1: Comparison of different compilers that use an RL solution to optimize code

regular computations, whereas the other frameworks target more diverse optimization scenarios without employing this abstraction.

- **Cost Model Integration:** Only *Tiramisu* integrates a cost model, allowing it to predict optimization outcomes before execution, this allows it to reduce the agent training time significantly.
- **Role in the RL Workflow:** *Tiramisu*, *Halide*, and *MLIR-RL* utilize RL agents to directly optimize code transformations. In contrast, *PolyGym* and *CompilerGym* act as RL environments, enabling users to implement and experiment with custom agents.
- **Algorithm Utilized:** The use of *Proximal Policy Optimization (PPO)* in *Tiramisu*, *Halide*, and *MLIR-RL* highlights its efficacy in learning stable and robust optimization strategies. The absence of predefined algorithms in *PolyGym* and *CompilerGym* emphasizes their role as platforms for RL experimentation.
- **MDP Modeling:** All frameworks model their tasks as Markov Decision Processes (MDPs), reinforcing RL's suitability for sequential decision-making in compiler optimization.
- **Action Space:** *Tiramisu*'s fixed action space of 53 actions is designed specifically for its polyhedral optimization tasks. The other frameworks adopt dynamic action spaces, adapting to various tasks and environments, which enhances flexibility but may require additional configuration.
- **State Representation:** Each framework employs a distinct approach:
  - *Tiramisu* and *MLIR-RL* use observation vectors to capture code features.

  - *Halide* represents state with its directives and parameters, reflecting its domain-specific design.
  - *PolyGym* and *CompilerGym* focus on points in graphs and Intermediate Representations (IR), respectively, to support broader optimization tasks.

- **Reward Function:** The universal use of speedup as the reward metric highlights the shared objective of enhancing execution performance across all frameworks.

This comparative analysis illustrates the diverse strategies employed in RL-based compiler optimization. *Tiramisu* and *PolyGym* emphasize the polyhedral model for structured computations, while other frameworks provide versatile platforms for experimenting with various RL techniques. The reliance on PPO demonstrates its effectiveness, while the modularity of *PolyGym* and *CompilerGym* makes them valuable resources for RL research in compiler optimization.

Conclusion

This chapter has presented a comprehensive review of reinforcement learning approaches in compiler optimization, examining five major frameworks: Tiramisu, CompilerGym, Halide, PolyGym, and MLIR-RL. The frameworks demonstrate various approaches to the core RL components—state representation, action spaces, and reward functions—while sharing common objectives in performance optimization. Despite their differences in implementation, all frameworks successfully model compiler optimization as a Markov Decision Process, with PPO emerging as a preferred algorithm for several implementations. Through this comparative study, we have identified key design decisions that influence framework capabilities, such as the trade-off between specialized optimizations and general applicability. This analysis provides valuable insights highlighting both successful approaches and areas for potential improvement.

Chapter 4

Design & implementation

4.1 Motivation

Accelerating the execution of neural network models is crucial for deploying efficient machine learning applications. While MLIR provides a flexible and modular framework for building domain-specific compilers, the sheer complexity of transformation choices makes manual optimization challenging and time-consuming. Reinforcement learning (RL) has recently emerged as a promising approach to automate this process by exploring sequences of compiler transformations.

Existing RL-based approaches implemented in MLIR, however, are limited in scope. They operate at the level of individual operations without accounting for the broader structure of full neural network programs. As a result, they fail to capture important interactions between operations that significantly affect runtime performance.

This project addresses this limitation by extending the MLIR-RL system (Bendib et al. 2024) to support the optimization of entire neural network programs through a richer set of transformations. Our contributions include the integration of the **fusion transformation**, a redesigned representation of the program state, a new action space, a dataset tailored for full-program optimization, and a mechanism to specify optimization decisions across entire neural network graphs.

Our main goals are:

- **Support full neural network optimization:** Extend the existing RL system to operate on complete programs, rather than isolated operations.
- **Supports a wider range of operations:** Allow the RL agent to optimize more linear algebra and general operations found in neural networks.
- **Introduce fusion transformation:** Enable the RL agent to apply fusion as part of its action space, allowing more complex and effective optimizations.
- **Reduce execution time:** Improve runtime performance by learning and applying efficient transformation sequences.

In the following sections, we will be modeling the various changes made to the reinforcement learning system to support optimization of full neural network code. This includes defining the new representation of states and actions, adapting the environment, and incorporating the fusion transformation to enable the agent to make effective optimization decisions across complete MLIR programs.

4.2 Dataset generation

We curated a new dataset specifically for our task. Since our objective is to optimize full neural network code, we required a dataset that represents sequences of Linalg operations similar to those found in real neural network models. To construct this dataset, we adopted two approaches. The first approach involves randomly synthesizing sequences of Linalg operations. The second approach consists of extracting widely used computation blocks from well-known neural network models such as ResNet and residual (He et al. 2016) blocks.

In order to ensure that the training remains computationally feasible, we imposed a limit on the sequence length. We set this limit to `L`=5, which provides a balance between keeping the training time reasonable while still allowing the agent to learn how to handle multiple operations simultaneously.

We will detail the generation of both datasets in the following sections.

4.2.1 Synthesized sequences

For this approach we generate a random sequence of length `L`=5. Each operation in the sequence takes as input the output of the previous operation, which ensures that the resulting sequence forms a meaningful chain of computations. The operations are generated with random shapes and are randomly selected from the following set of Linalg (linear algebra dialect of MLIR) operations: `add`, `matmul`, `relu`, `conv_2d` and `pooling` which are the same operations supported by the previous system (Bendib et al. 2024), while also adding two new operations widely used in neural networks models which are `sigmoid` and `softmax_2d` to allow the agent to familiarize itself with more activation functions which will greatly help the agent when faced with real world programs.

4.2.2 Blocks of NN models

We have manually extracted from the resnet (He et al. 2016) model its repeated block which is known as the resnet block. It consists of convolution and maxpooling operations with relu and batch normalization interleaved within. It's significantly different than the synthesized programs that is because, the data-flow is not as predictable compared to the former. This will give the model a more challenging task for when it decides to apply the fusion transformation and will be a great example for real world neural networks to optimize.

We have also included a simple residual (He et al. 2016) block which can be represented by this equation

$$F(x) + x$$

where F is any operation and we have opted to choose matmul + relu as our F for simplicity.

We also included classical blocks such as the linear block

$$activation(x * W + b)$$

with relu and sigmoid as activation layer. This block is very crucial to our task as it is regarded as the founding block of neural network and deep learning. we also included a simple convolution block made of a conv_2d operation followed by a relu activation.

It is also important to note that during dataset generation, we execute each generated code sequence to verify its correctness and to record its base execution time, which will later be used during the training process. As the process is computationally intensive and time-consuming, we chose to generate a dataset of 7887 for the agent to train on. The following table shows the distribution of each program types included in the final dataset.

program type	number of instances
single operations	298
NN blocks	188
synthesized programs	7401

Table 4.1: Table of each program type and its number of occurrences in the dataset

4.3 Actions

In a reinforcement learning system, an action defines what the agent decides to apply and by which the environment transitions from the current state S_t to a new state S_{t+1}. In our context, as defined in (Bendib et al. 2024) an action corresponds to applying a specific transformation (possibly with parameters) to an operation, moving to the next operation, or choosing to terminate the optimization schedule.

The original system (Bendib et al. 2024) supported the following code optimization transformations: **Tiling+Parallelization**(TP), **Tiling**(T), **Interchange**(I),**Vectorization** (V). To support optimization of full neural network programs, we introduce two additional actions:

- **Tiling + Fusion (TF):** Applies loop tiling to the current operation and attempts to fuse it with its current producer, if fusion is semantically valid.In this context, the *producer* is an operation that generates data consumed by another operation, while the *consumer* is the operation that uses this data.

  In the MLIR framework, applying fusion on its own is not possible: the Transform dialect requires the consumer operation to be tiled first before fusion can be applied. This constraint led us to combine tiling and fusion into a single transformation (similar to the approach done by (Bendib et al. 2024), where tiling and parallelization were also grouped).

  This action allows the agent to perform cross-operation optimizations that improve performance by reducing memory traffic and increasing locality. The number of variants for this action in the action space is $M \times M \times \ldots \times M$ (N times), where M is the number of tile size options per loop and N is the number of loops.

- **Next:** It is one of the two terminal operations (which includes vectorization) it marks the schedule of the current operation as finished and as a result the agent stops applying transformations on the current operation and the environment will move directly to the next one. This action will give the agent more flexibility which helps it avoid applying suboptimal schedules if the agent judges that applying any other transformation will hurt the final execution time.

Together, these actions let the agent create more flexible optimization schedules that work across multiple operations. This expanded action space helps the system make better decisions by considering both local changes and the overall structure of the program.

4.4 State and Observation

To enable the reinforcement learning agent to make informed optimization decisions, MLIR programs must be encoded into a structured format that captures the essential computational characteristics and dependencies between operations. Our approach leverages the Abstract Syntax Tree (AST) structure -inspired by (reference tiramisu)- inherent in MLIR to construct a hierarchical tree representation that preserves the program's structural information while providing the numerical features necessary for the RL agent.

Since our input consists of a sequence of Linalg operations, directly lowering the MLIR code to the affine level is not straightforward. This is primarily due to the absence of a one-to-one correspondence between individual Linalg operations and their affine representations. To address this, we adopt a strategy where each Linalg operation is isolated and lowered independently into the affine dialect. By performing this lowering on a per-operation basis, we ensure that the resulting affine code accurately captures the semantics of each Linalg operation without introducing dependencies or ambiguities that could arise from a global lowering. Once the affine representation of the operation is obtained, we proceed to extract its feature representation, which serves as the input to our learning agent for optimization tasks.

After lowering the Linalg operation to its affine representation, we construct a tree representation of its loop structure, where each node in the tree corresponds to a computation vector for the operations at that loop level as shown in the illustration 4.1. We build such a tree for both the consumer and the producer, and together these two trees constitute the observation that will be provided to the agent.

The computation vector is a concatenation of several features extracted from the operation, namely:

- **Loop information:** By lowering the Linalg operation to the affine dialect, we are able to extract information about the loop structure—specifically, the lower bound, upper bound, and step size. In our case, the lower bound and step are always set to 0 and 1 respectively, so we discard them and retain only the upper bounds of the loops as relevant information.
- **Load access matrices:** Within the loop nests, data is loaded from one or more vectors using loop iterators. To track which iterators are used to index which vectors, we represent each load operation with an access matrix. This matrix has shape $(D, N+1)$, where D is the number of dimensions (indices) of the loaded vector and N is the number of loop iterators. The last column accounts for constant offsets.
- **Store access matrix:** Similar to load access matrices, each store operation is represented using a $(D, N+1)$ matrix. Since each Linalg operation performs a single store, we have one store access matrix per operation.
- **Operation counts:** We also record the number of arithmetic operations within the loop nest, including addition, subtraction, multiplication, division, logarithm, and exponential operations.

To ensure that the computation vector has a fixed size for all types of operations and loop nest depths, we specify maximum sizes for each component:

- **Loop information** (N): We fix the maximum number of nested loops per operation to 7. If an operation has fewer than 7 loops, we pad the remaining entries.
- **Load access matrices** (L): The maximum number of load operations is set to 7, based on the maximum observed in our dataset. Operations with fewer loads are padded accordingly.
- **Maximum number of indices** (D): We set the maximum number of indices in an access matrix to 7, corresponding to the highest dimension observed for vectors in our dataset. If fewer indices are present, we pad the matrix.

Under these assumptions, we can compute the shape of the computation vector as follows:

$$\underbrace{N}_{\text{loop upper bounds}} + \underbrace{L \times D \times (N+1)}_{\text{load access matrices}} + \underbrace{D \times (N+1)}_{\text{store access matrix}} + \underbrace{6}_{\text{operation counts}} .$$

The environment state also maintains additional information that is relevant for decision-making, such as the operation tags, the action history, the current execution time, the initial (base) execution time, among others. These state variables assist the agent in the decision-making process, for example by masking invalid actions or terminating schedules when necessary.

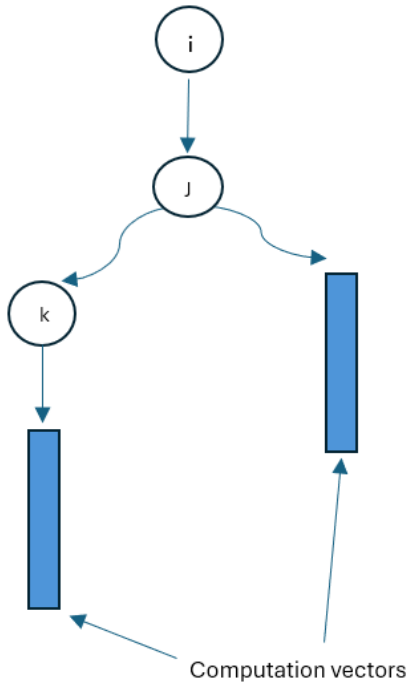


Figure 4.1: Example of an AST representation with the computation vectors

4.5 Environment

The environment is the main component of a reinforcement learning environment; it defines the setting in which the agent receives inputs, takes actions, receives rewards, and transitions to new states.

A reinforcement learning environment has two main functions: a reset function and a step function. The reset function initializes the environment to a starting state for a new episode, while the step function executes the action selected by the agent and returns the new observation, reward, and other information (such as whether the episode has ended) that the agent uses for subsequent decisions.

In the following sections we will describe each function:

4.5.1 Reset function

As mentioned previously, the reset function is responsible for initializing the environment to a starting state and returning the initial observation. In our context, this involves randomly selecting an MLIR code sample

to optimize and resetting all relevant information to its initial state.

The reset function sets the first operation to be optimized as the last Linalg operation in the code. This means that the agent begins optimization from the last consumer in the computation graph and proceeds backwards toward its producers, effectively traversing the operations in reverse order.

This design choice is motivated by the fact that many compiler optimizations, particularly those related to fusion, tiling, and scheduling, depend not only on the properties of an operation itself but also on how its results are used. Starting from the consumers allows the agent to take into account the actual usage context of each operation before deciding how to optimize its dependencies. (this needs to be double checked maybe add a source)

Each Linalg operation in the MLIR code is assigned a unique identifier. Using MLIR analysis tools, we extract all producer-consumer relationships, which are represented in the form: `producer_tag -> consumer_tag`. This allows us to efficiently determine the set of producers corresponding to the operation currently being optimized.

Algorithm 5 describes the procedure of the reset function.

Algorithm 5 Environment Reset

function `reset`(*idx*)
Input: Optional index `idx`
Output: Initial state, observation
if *`idx` is provided* **then**
  Set `bench_index` to `idx`
else
  Select a random benchmark index and assign it to `bench_index`
Load `bench_name` and `benchmark_data` from `benchmarks_data[bench_index]`
Set `operation_index` to last operation in benchmark
if *operation has producers* **then**
  Set `producer_tag` to first producer
else
  Set `producer_tag` to `None`
// initialize State
state = new State()
state.benchmark_data = benchmark_data
state.operation_tag = `operation_tag`
state.producer_tag = `producer_tag`
state.action_mask = initialize_action_mask()
state.action_history = []
`obs = build_observation(state)`
return `state, obs`

4.5.2 Step Function

The step function is the most critical component of our environment. It takes the action returned by the agent, applies it to the current state, and returns both a reward—which defines the agent's learning policy—and the next state to be used by the environment. This function is also responsible for applying all code transformations and updating the information stored in the previous state. We have dedicated significant

effort to curating this function, testing various configurations, and evaluating which settings yield the most optimal results.

The function begins by applying the code transformation selected by the agent for the current operation. If the chosen action is fusion, the fusion transformation is invoked using information about the consumer and its producer. If the fusion is successful—meaning the transformation was semantically valid and applied correctly in the code, resulting in a new code—we update the producer information—and the current consumer operation has multiple producers, we move to the next one. If it has only one producer, or if all producers have already been fused, we mask the fusion action in subsequent steps to prevent the agent from selecting it again.

We have also observed that combining fusion and vectorization generally yields better results. Therefore, when the agent selects fusion, we mask all other transformations and allow only vectorization to be selected in the next step. It is important to note that vectorization is treated as a terminal action; once the agent selects vectorization, the schedule is terminated, as no further transformations can be applied.

Once the agent has completed the schedule for the current operation, it selects the *Next* action and moves to the next operation to be optimized by traversing backwards through the computation graph. This process is repeated until the agent has traversed the entire computation graph.

Algorithm 6 describes the procedure of the step function.

Algorithm 6 Step function of the environment

function `Step`(*state*, *action*)
Input : Current state, selected action
Output: Next state, reward
Apply the selected transformation to the current operation
if *action is* `Fusion` **then**
  if *fusion successful* **then**
    if *multiple producers remain* **then**
      Move to next producer
    else
      Mask fusion action for next steps
else if *action is* `Vectorization` *or actions is* `No Transformation` **then**
  Mark schedule as terminated (terminal action)
Compute reward based on the execution time of applied transformations
Update transformation history and state information
if *agent selects* `Next` *action or schedule is terminated* **then**
  Move to the next producer operation in computation graph
  if *all operations traversed* **then**
    Mark episode as finished
return updated state, reward

4.6 Policy

In reinforcement learning, the policy plays a fundamental role in defining how the agent interacts with its environment. Essentially, it determines which action the agent should take in any given state. Formally, a policy is a function π that maps each state $s \in S$ to a probability distribution over the set of possible actions. In other words, $\pi(a|s)$ gives the probability that the agent selects action a when it finds itself in state s, i.e., $\pi(a|s) = P(A_t = a \,|\, S_t = s)$.

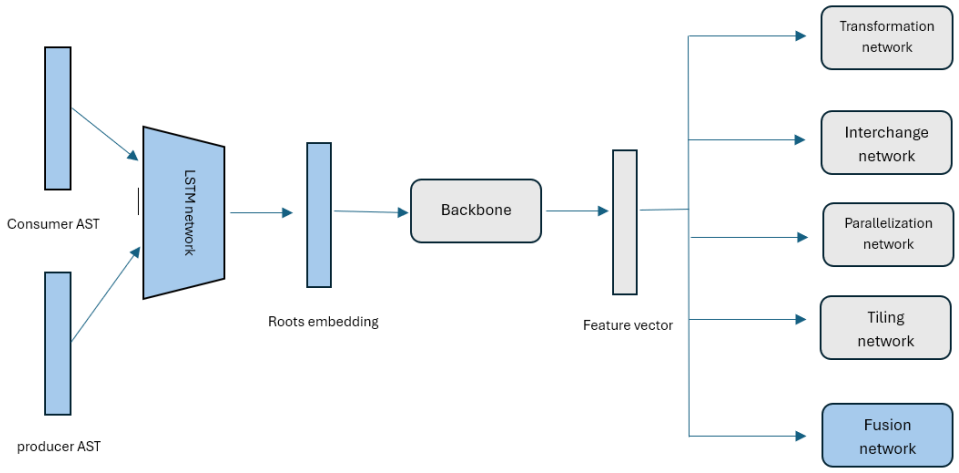


Figure 4.2: Updated policy network with the recursive loop embeddings and the fusion head

This policy acts as the agent's strategy, guiding its behavior throughout the learning process with the goal of maximizing the total reward accumulated over time. The ultimate aim of reinforcement learning is to discover an optimal policy π^*, which leads to the highest expected return starting from any state. Associated with any policy π is its value function $V^\pi(s)$, which estimates the expected cumulative reward when the agent starts from state s and continues to act according to π.

In deep reinforcement learning, the policy is typically modeled using a trainable neural network, which learns to approximate the optimal policy by adjusting its parameters based on interactions with the environment. In our context, we updated the policy network by adding a recursive loop embedding network that takes as input the AST structure of both the producer and consumer operations and generates a new embedding that becomes the input to our backbone, which after processing passes it into the policy network and value network as shown in figure 4.2.

- **Recursive loop embedding:** This is the part responsible of encoding the AST representation of our Linalg operations into a single vector that can be used later by the rest of the network, it recursively traverses the AST structure starting from the leaves. It first starts by encoding the computation vectors at the most nested levels using the loop embedding unit as described in figure 4.3 , then recursively works it's way back to the parent node. the loop embedding unit is composed of two LSTM's unit one that encodes loop embedding generated from the child loop levels and the other one encodes the computation vector of the current level, it then concatenates both the embeddings and we pass it into a feed forward network as illustrated in the figure 4.4.

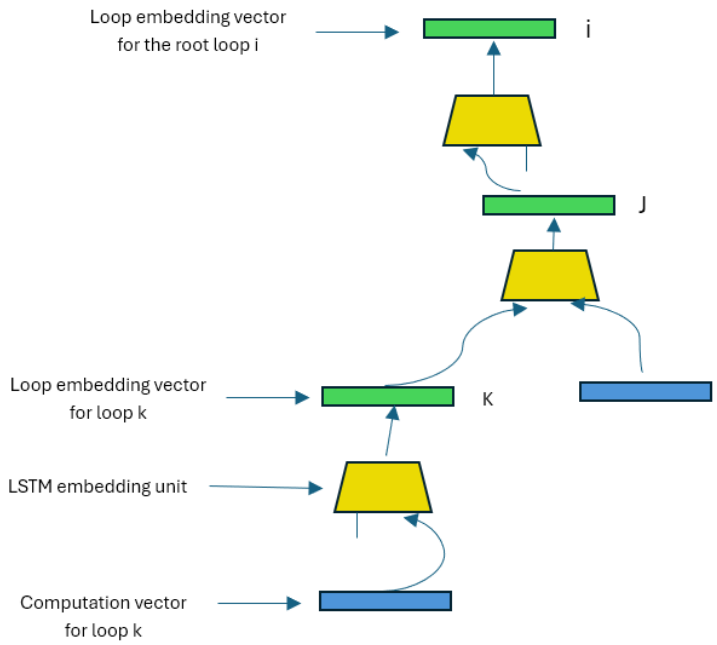


Figure 4.3: Processing of the program presented in figure 4.1 using the recursive loop embedding method

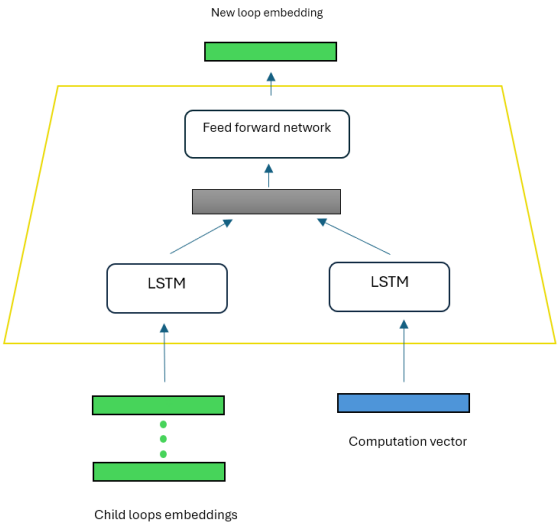


Figure 4.4: LSTM embedding unit that generates the new loop embeddings

The following algorithm 7 summarizes the procedure used to generate the loop embeddings for our AST structure.

Algorithm 7 Recursive Loop Embedding

function `get_hidden_state(node)`
Input : Root node of the Abstract Syntax Tree (AST)
Output: The final embedding for the input node
if *node has children* **then**
  Recursively compute embeddings of child nodes
  Aggregate child embeddings into a single tensor
  Pass aggregated child embeddings through the `children_nodes_LSTM` to get `child_embedding`
else
  Use default child embedding vector
if *node has computation vector* **then**
  Pass computation vector through the `computation_LSTM` to get `comp_embedding`
else
  Use default computation embedding vector
Concatenate the `child_embedding` and `comp_embedding` and Pass it through the feedforward network
return final embedding

- **Tiling + Fusion Head:** As mentioned previously, to apply fusion we must first apply tiling to the loop nest. Therefore, we need to determine the tiling parameters for the fusion transformation. Each loop has $(M+1)$ possible tiling sizes: the first one is 0, which represents no tiling, and the remaining M correspond to actual tile sizes. Similar to the Tiling + Parallelization branch, we use two fully connected layers: the first has 512 nodes, and the second has $N \times (M+1)$ nodes, where N is the number of loops, since we must determine the tile size for each loop. The output is reshaped to $(N, M+1)$, and we sample a tile size for each loop as illustrated in Figure 4.5.

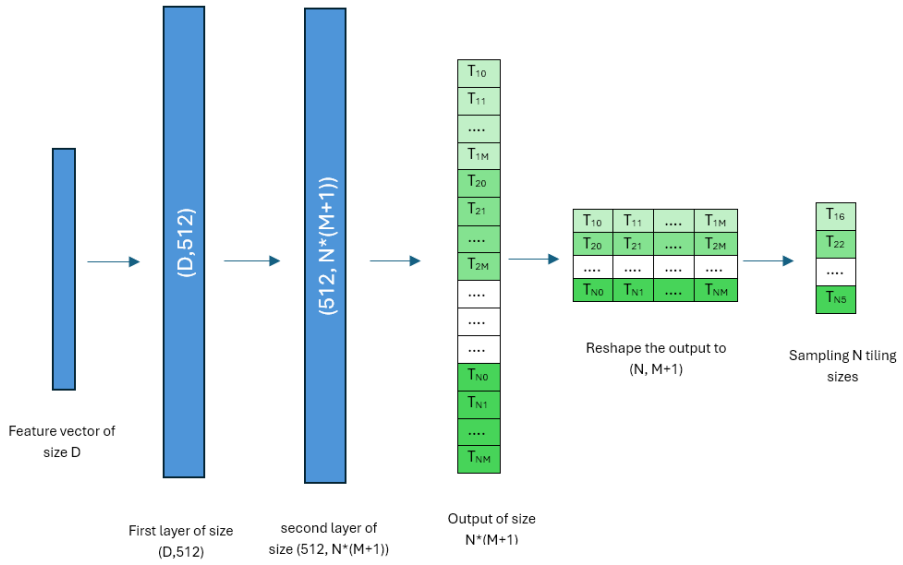


Figure 4.5: Tiling + Fusion policy network

4.7 Reward

The Agent-Environment hinges on one very crucial element, which is the reward; it conveys to the agent the performance of its actions in quantitative measures, which let him in turn adjust it for the future and eventually learn from it. It becomes clear that a well defined reward system is a defining factor for a good RL system.

Continuing on the previous system's results (Bendib et al. 2024) where they showed that the final reward system where the agent is given the sum of the episode's entire reward gives more performance benefits compared to rewarding the agent each time it applies an optimization (Immediate reward), when extending this system to accept full code optimization where multiple operations are to be expected, we can follow an extended final reward system where we only reward after the agent is *done* optimizing the entire code but we fear that this method delays the reward too much and does not convey the true effectiveness of the transformation to the agent.

As a result, we opt to choose a more balanced approach that incorporates the strength of **final reward** but in a more similar context as the experiments of the previous system. The agent will receive the reward only once it finishes optimizing an operation and just before transitioning to the **next** operation.

We calculate the reward with respect to the speedup achieved by the agent when optimizing the code; we calculate the execution time of the new code (after applying the optimization transformation) and compare it to the base execution time (recorded in the dataset). We reward the agent with a positive value if it achieves a speedup greater than 1; otherwise, we penalize the agent by providing a negative value if it underperforms or if the agent outputs an invalid schedule (while effectively terminating the episode).

$$r = \begin{cases} log_{10}(\frac{base\ execution\ time}{new\ execution\ time}) & \textbf{if}\ \ the\ schedule\ is\ valid \\ \text{-5} & \textbf{if}\ \ applying\ the\ transformation\ fails \\ \text{-20} & \textbf{if}\ \ an\ error\ while\ execution\ occurs \end{cases}$$

It is important to note that -5 and -20 are values chosen by the team behind the previous system, and we did not see any benefits of altering them.

4.8 Implementation

In this section we'll discuss the different tools and frameworks we used to implement our design. We'll also detail how the communication between our python code and the MLIR compiler is being established and finally we'll detail some technical parts about our training process.

4.8.1 Tools and frameworks

To support extensive experimentation and efficiently explore various configuration options in our solution, we assembled a set of tools that streamlined key aspects of our workflow. These tools aided in building the environment and neural networks, tracking experimental progress through visualizations, and comparing results across runs. The main tools used in our research are listed below.

Pytorch

PyTorch (Paszke et al. 2019) is an open-source deep learning framework developed by Facebook's AI Research lab. It offers a flexible platform for building and training neural networks, with dynamic computation graphs and GPU acceleration capabilities that make it particularly well-suited for research. In our work, PyTorch was used to implement the models guiding the reinforcement learning agent, enabling rapid prototyping and efficient training.

Neptune

Neptune (Neptune.ai 2024) is a metadata management tool for MLOps, developed by Neptune Labs. It is designed to support the tracking and organization of machine learning experiments, promoting collaboration, reproducibility, and transparency in research workflows. Neptune enables users to log experiment metadata, monitor training metrics in real time, and compare different model versions and configurations. This functionality is especially valuable in reinforcement learning projects, where managing a large number of experiments with varying hyperparameters and settings is essential for systematic evaluation and progress tracking.

MLIR

MLIR (C. Lattner et al. 2021) is a compiler infrastructure developed by LLVM (Chris Lattner et al. 2004) with the goal of reducing the complexity involved in building machine learning frameworks and domain-specific compilers. It introduces a unified intermediate representation that enables optimization and transformation across multiple levels of abstraction. MLIR supports the creation of custom dialects and transformation passes, making it highly extensible and particularly well-suited for optimizing the kind of high-performance code generated in modern machine learning workflows.

4.8.2 MLIR Python Bindings

MLIR Python bindings (C. Lattner et al. 2021) provide Python developers with direct access to the MLIR framework from within Python. These bindings make it possible to script and automate various compilation and optimization tasks without writing C++ code. This capability is particularly valuable in our research,

as it allows seamless integration with Python-based tools and libraries, facilitating the efficient development, testing, and deployment of compiler optimizations and reinforcement learning models.

4.8.3 Computing Infrastructure

The experiments and training procedures in this work were conducted on the **Jubail High Performance Computing (HPC) cluster** at **New York University Abu Dhabi (NYUAD)**. Jubail is a medium-sized Linux-based HPC system launched in March 2022 and hosted at the NYUAD Data Center in Saadiyat. It supports large-scale computational research through powerful CPU resources by utilizing the SLURM workload manager.

While the cluster comprises **269 CPU nodes** (totaling approximately **33,984 CPU cores**), all training in this work was performed on a single CPU node.

- **Processor:** AMD EPYC 7742 (64-Core, 2.25 GHz)
- **Cores per node:** 128 (2 $\times$ 64-core CPUs)
- **Memory per node:** 512 GB
- **Memory per core:** 3.75 GB

4.8.4 Training Acceleration via Caching

The training process for our reinforcement learning agent involves repeatedly applying transformation sequences, compiling the resulting MLIR code, and measuring its execution time. This compile-and-run cycle is the primary bottleneck, consuming the vast majority of the training time. Since the agent may explore the same transformation schedule for a given program multiple times, re-executing these evaluations is redundant and inefficient.

To accelerate training, we implemented a persistent caching mechanism that stores the measured execution time for any unique, transformed program. The core of this system is a key-value store implemented as a JSON file.

Key generation via hashing: A crucial design choice was how to uniquely identify a given transformed program to use as a cache key. Using the raw MLIR code as a key is impractical due to its potentially large size and variable length. Instead, we employ a stable cryptographic hash function (SHA-256) to generate a fixed-size, unique fingerprint for each code variant. This approach provides several advantages:

- **Uniformity:** All keys have a constant, short length, which is efficient for storage and lookups.
- **Uniqueness:** The probability of two different programs producing the same hash (a collision) is astronomically low, making it a reliable identifier.
- **Stability:** The hash is deterministic, ensuring that the same code will always produce the same key across different training runs, allowing the cache to be reused.

It is crucial to note that the cache is not pre-populated; rather, it is built incrementally throughout the training process. Consequently, the initial training run operates in a *cold start* mode, where every new transformation schedule results in a cache miss and incurs the full cost of compilation and execution.

However, the benefit of this approach is cumulative. Each subsequent training run leverages the cache built by all previous runs. As the agent explores the optimization space, the cache grows, and the amortized

cost of evaluating a schedule decreases significantly. This dynamic becomes particularly effective in the later stages of training, where the agent begins to converge and frequently revisits promising schedules. As a result, the cache hit rate increases, leading to a substantial acceleration of the overall training process.

4.9 Conclusion

This chapter detailed the design and implementation of our extended reinforcement learning system for optimizing full neural network programs in MLIR. We began by motivating the need to move beyond single-operation optimization to capture the complex interactions present in complete computational graphs.

The core contributions described include:

- A new dataset composed of synthesized sequences and real-world neural network blocks to train the agent on multi-operation programs.
- An expanded action space featuring **Tiling + Fusion** transformation, enabling the agent to perform inter-procedural optimizations.
- A hierarchical state representation based on the program's AST, which provides the policy network with rich structural and computational features of both consumer and producer operations.
- A redesigned environment that traverses the program graph from consumer to producer.
- An adapted policy network architecture, featuring a recursive loop embedding model to effectively process the new AST-based state.

We also outlined the implementation details, including the software frameworks like PyTorch and MLIR, the use of caching for accelerating training runtime, and the HPC infrastructure that supported our experiments. By establishing this comprehensive system, we have laid the groundwork for training an RL agent capable of learning sophisticated, whole-program optimization strategies. The following chapter will present the experimental results and evaluate the effectiveness of this approach.

Chapter 5

Experiments and Results

This chapter details the comprehensive set of experiments conducted to evaluate our reinforcement learning agent for compiler optimization. The core challenge in this domain lies in navigating a vast and complex search space of possible transformation sequences, where optimal decisions are highly context-dependent. Traditional compilers rely on hand-tuned heuristics, which can be brittle and struggle with new or unconventional program structures. Our research is motivated by the hypothesis that a learning-based agent can discover more effective, data-driven optimization strategies. To test this, our evaluation is designed to measure the agent's capabilities systematically, beginning with fundamental transformations and concluding with full-scale, real-world deep learning models. The objective is to present a clear and thorough picture of the agent's current strengths, particularly its adaptability, as well as its limitations when compared to mature, production-grade systems.

We begin our analysis by focusing on **operator fusion**, a critical compiler optimization that reduces memory overhead by merging computational kernels. This section serves two purposes: first, to confirm that the agent can successfully learn to apply this powerful transformation, and second, to illustrate the inherent complexity of the task. Through targeted experiments and a manually-constructed case study, we demonstrate that the decision to fuse is not always beneficial, as it can conflict with other optimizations like parallelization. This highlights a classic compiler dilemma and underscores the necessity for an intelligent agent that can navigate such non-obvious trade-offs, providing a strong motivation for our learning-based approach.

Following this, we examine the agent's training dynamics and the associated computational costs. An analysis of the learning curves for value loss, policy loss, entropy, and cumulative reward offers insight into the stability and convergence of the training process. We then proceed to evaluate the agent's learned policies on individual, high-impact operations: **matrix multiplication** and **convolution**. These kernels are the computational workhorses of most neural networks, and their efficient execution is paramount. This evaluation serves as a crucial intermediate step to validate that the agent can effectively optimize these foundational building blocks before we assess its performance on more complex, interconnected graphs.

The chapter concludes with an end-to-end benchmark on three neural network models: **VGG, ResNet, and MobileNet**. We compare our agent's results against the highly-optimized PyTorch framework. This final evaluation highlights our agent's potential, especially on modern architectures, while also identifying its current limitations and providing clear directions for future work.

5.1 Metrics

Throughout this chapter, we aim to evaluate our method by measuring the *execution time* of programs in ms before and after letting the agent decides the schedule to apply to the programs. We also will be calculating the *speedup* generated by the agent's schedules. Speedup are defined as follow:

$$Speedup = \frac{old\_execution\_time}{new\_execution\_time}$$

a value of speedup strictly bigger than 1 means that the schedule is good and it improved the execution time. The smaller the resulting *execution time* is the better while the bigger *speedup* is the better and means that the agent is performing better.

5.2 Training on Linear sequences

The objective of this training is to evaluate the effectiveness of the fusion transformation and determine whether the agent is capable of learning to fuse simple blocks of operations. For this reason, we trained the agent on a dataset composed exclusively of linear blocks of the form activation$(x * W + b)$.

The following table presents the hyperparameters used for the PPO agent.

Hyperparameter	**Value**
Learning rate	0.001
Discount factor (gamma)	0.99
Entropy coefficient	0.01
Value function coefficient	0.5
PPO clip parameter	0.2
Number of epochs for PPO	4
Mini-batch size	64
GAE lambda	0.95

Table 5.1: PPO hyperparameters

5.2.1 Results

Table 5.2 presents the performance results of applying different transformation schedules to the `add` operation. The old and new execution times are compared, and the resulting speedup is reported. As shown, the combination of tiling, fusion, and vectorization leads to a significant improvement in performance.

Schedule	**Old_exec**	**New_exec**	**Speedup**
Tiled & Fused & Vectorized	8.42 ms	0.8 ms	10
Tiled	8.42 ms	8.97 ms	0.9
Tiled & Parallelized & Vectorized	8.42 ms	8.98 ms	0.9

Table 5.2: Performance comparison of different schedules applied to the add operation.

5.2.2 Interpretation

The results in Table 5.2 demonstrate the impact of different transformation schedules on the execution time of the `add` operation. We observe that the schedule combining tiling, fusion, and vectorization significantly outperforms the other configurations, achieving a 10x speedup compared to the baseline. This indicates that the agent successfully learned to apply fusion in a way that groups the `add` operation with the preceding `matmul`, enabling the compiler to generate more optimized code.

In contrast, applying only tiling, or tiling with parallelization and vectorization, yields negligible improvements or even slight regressions. This suggests that fusion plays a crucial role in performance enhancement, particularly when applied in conjunction with other optimizations like vectorization.

5.3 Case Study: The Conditional Benefit of Fusion

While previous results show that fusion can be highly effective, it is not a universally optimal transformation. The best optimization strategy often depends on a complex interplay between transformations applied across an entire computation graph. To illustrate this, we manually constructed a case study analyzing a `linalg.matmul` operation followed by a `linalg.add` similar to what we had previously.

We compare two distinct, manually-defined optimization schedules. The goal is to understand the performance trade-offs a heuristic or a learning agent would face. The baseline execution time for the unoptimized sequence was approximately 931 ms.

Schedule	Transformations Applied	New Time	Speedup
Schedule A	`Add`: **Fusion** + Vectorization `MatMul`: Vectorization	75.8 ms	12.3x
Schedule B	`Add`: Tiling `MatMul`: Parallelization + Vectorization	53.9 ms	**17.3x**

Table 5.3: Performance comparison of two schedules for a `MatMul + Add` sequence. The baseline execution time is 931 ms.

5.3.1 Interpretation

The results in Table 5.3 highlight a classic compiler optimization dilemma.

Schedule A represents a strong, intuitive choice. Applying fusion to the `add` operation eliminates the memory-access overhead of the intermediate tensor, leading to a substantial **12.3x speedup**. This confirms that fusion is a powerful optimization for improving memory locality.

However, **Schedule B** reveals a more optimal, less obvious, path. By `not` fusing the operations, it becomes possible to apply a more aggressive parallelization and vectorization scheme to the computationally-heavy `matmul` operation. The performance gains from this enhanced parallelization outweigh the benefits of fusion, resulting in a superior **17.3x speedup**.

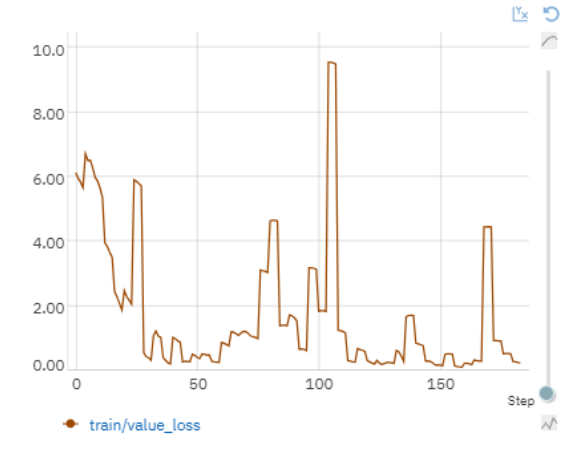
This case study demonstrates that the best transformation sequence is not always the one that applies the most locally-beneficial optimizations. The decision to fuse can inadvertently constrain other, more impactful

transformations like parallelization. Navigating these complex, non-linear trade-offs is extremely challenging for static heuristics. This is precisely the type of problem where a learning-based agent can provide significant value, as it can explore the vast search space and learn to identify these globally optimal, context-dependent strategies automatically.

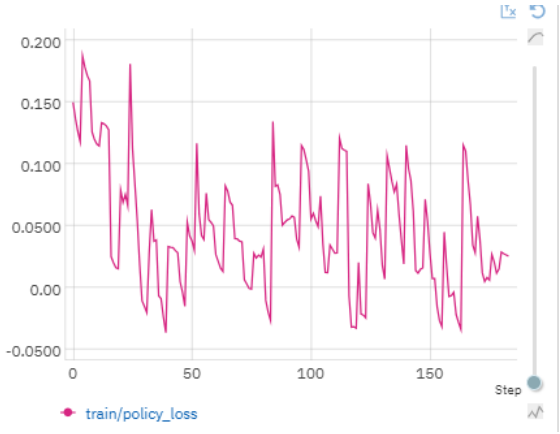
5.4 Training on sequences

In this section we'll focus on training the model on sequences of operations as mentioned previously in our Design chapter. These sequences consist of two types: synthesized and model-extracted blocks.

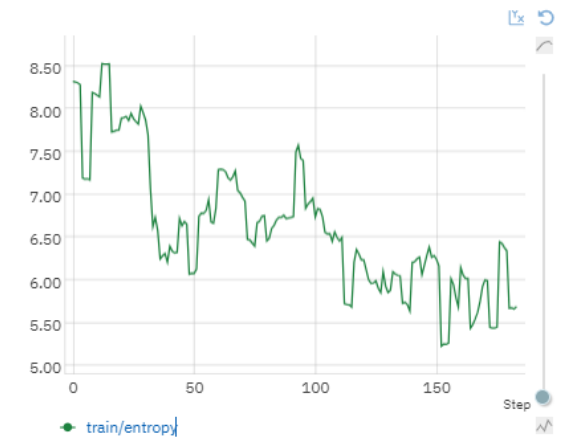
The following illustrations show the training graphs for our model:



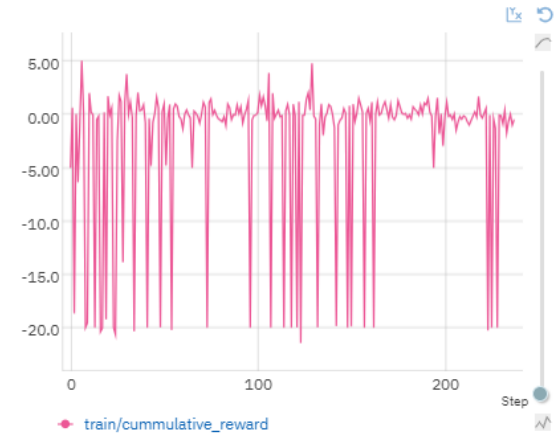
(a) Value loss



(b) Policy loss



(c) Entropy



(d) Cumulative reward

Figure 5.1: Model training results

Analysis

Figure 5.1 presents the evolution of key training metrics over time. Below is an analysis of each graph:

- **Value Loss (a):** The value loss shows significant fluctuations at the beginning of training, with several spikes, before gradually stabilizing. This indicates that the value function initially had difficulty estimating the expected returns accurately. As training progresses, the model improves its predictions, resulting in a lower and more stable loss.
- **Policy Loss (b):** The policy loss oscillates around zero without a clear upward or downward trend. This behavior is consistent with the PPO algorithm, which uses a clipped objective to prevent large policy updates. The fluctuations suggest that the policy continues to be updated moderately throughout training.
- **Entropy (c):** The entropy starts at a relatively high value and gradually decreases over time. This is expected, as high entropy during the early stages promotes exploration, while the gradual decline indicates that the agent becomes more confident and begins to exploit known strategies.
- **Cumulative Reward (d):** The cumulative reward shows high variance initially, with several negative values. Over time, it becomes more stable and increases in magnitude, suggesting that the agent is learning a more effective policy and achieving better performance.

Overall, the graphs reflect the typical learning behavior of a PPO agent. The model transitions from a phase of exploration and instability to one of more stable learning and policy improvement.

It is important to note the number of steps performed by the agent during training. The agent was trained for approximately 90 hours and completed only around 200 steps. This is a relatively small number for a reinforcement learning agent. The main reason for this low step count is that, unlike environments where a single operation is optimized, our setup involves optimizing sequences of five operations. Additionally, the environment performs an execution after each individual operation is optimized, further increasing the computational overhead and significantly slowing down the overall training process.

5.4.1 Evaluation on Single Operations

After training the agent on both blocks and sequences of operations, we proceed to evaluate its performance on individual operations before moving on to full neural networks. This intermediate evaluation step is crucial, as the ability to generate speedups on single operations serves as a strong indicator of the agent's capacity to perform well on more complex sequences.

If the agent fails to achieve performance improvements on isolated operations, it is unlikely to produce meaningful optimizations when applied to longer and more interdependent operation sequences.

We focus our evaluation on matrix multiplication (`matmul`) and 2D convolutions (`conv`) operations, as they are among the most computationally intensive in neural networks and offer substantial potential for optimization. These operations also frequently serve as performance bottlenecks in deep learning workloads, making them ideal candidates for targeted evaluation.

We will be using for the evaluation set a manually curated set of the most used matrix multiplication and convolution operations throughout state of the art neural networks, this set was compiled by the previous system's research team (Bendib et al. 2024).

Results

Figures 5.2 and 5.3 both show that the agent was able to find effective schedules resulting in speedups.

In Figure 5.2, we observe that most matrix multiplication (`matmul`) configurations benefited significantly from the learned transformations, with speedups reaching up to 20x in some cases. This demonstrates the agent's ability to identify and exploit optimization opportunities in dense linear algebra operations. Notably, the agent performs particularly well on larger shapes (e.g., 256×512×1024 and 256×1024×1024), which are typically more computationally intensive and offer greater potential for performance gains through scheduling.

Figure 5.3 presents the results for `conv2d` operations, where the speedups are more modest but still consistent. The agent achieves speedups ranging between 2x and 10x across various convolution configurations. This indicates that while convolution operations are generally harder to optimize due to their more complex memory and compute patterns, the agent is still able to generate effective schedules. The highest gains appear for mid-sized convolutions, which likely offer a good balance between computational density and optimization flexibility.

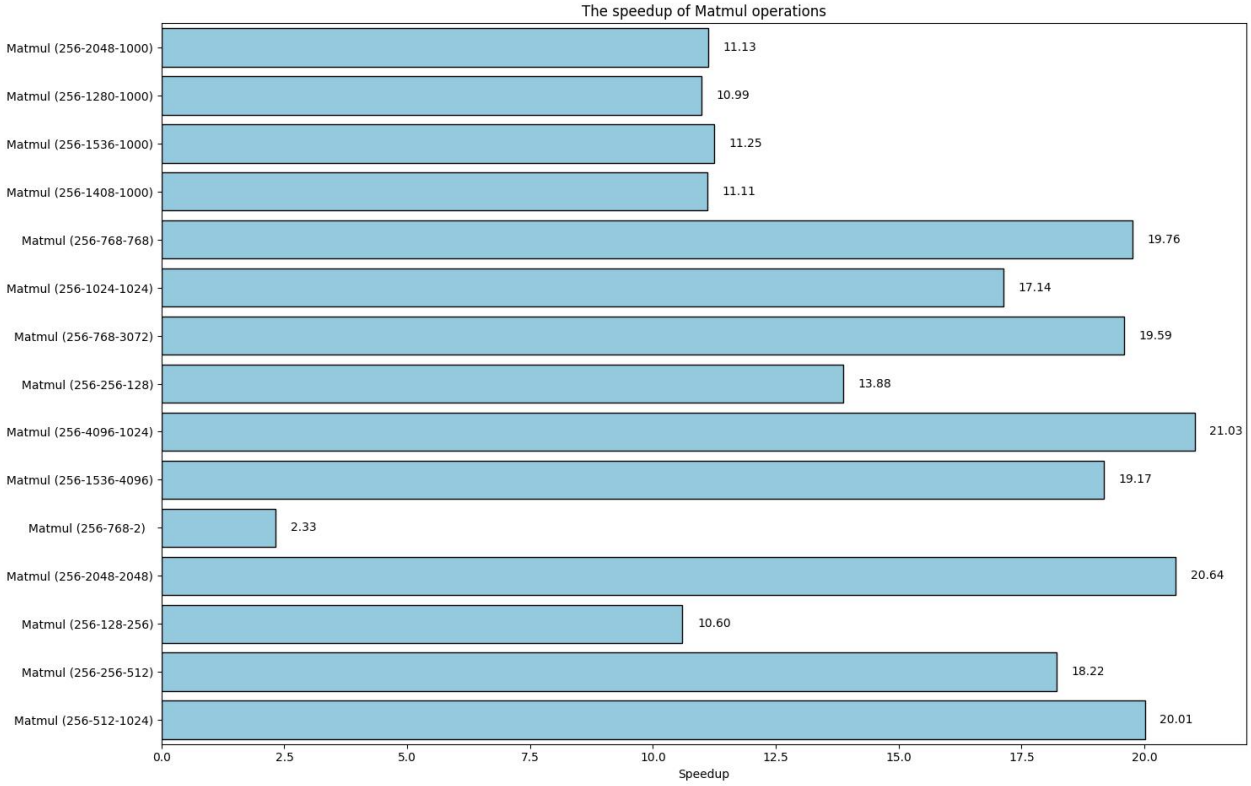


Figure 5.2: Speedups for different matmul operations

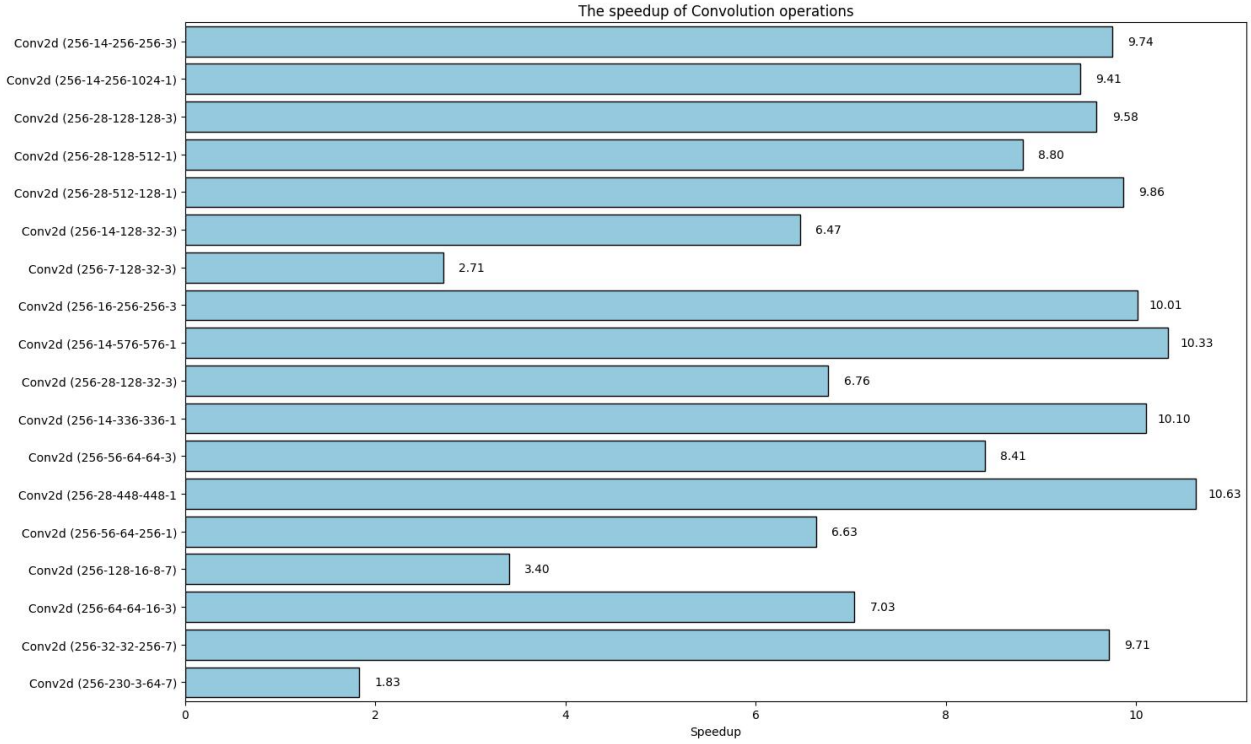


Figure 5.3: Speedups for different convolution operations

Conclusion

The evaluation on individual operations confirms the effectiveness of the agent in learning performance-enhancing schedules. Significant speedups on `matmul` and `conv2d` operations—particularly on larger and mid-sized configurations—demonstrate that the agent is capable of generalizing its learned optimization strategies beyond the training sequences. These results provide a strong foundation for extending the evaluation to full neural networks, where the ability to optimize single components is essential for achieving end-to-end performance gains.

5.4.2 Evaluation on Neural networks models

In this section, we evaluate the agent's performance on full neural network models to assess its ability to generalize optimization strategies beyond isolated operations or short sequences. The goal is to observe whether the schedules learned during training can translate into meaningful performance improvements when applied to real-world architectures.

We selected three widely used convolutional neural network (CNN) architectures for this evaluation:

- **VGG (Visual Geometry Group):** VGG is a deep convolutional network known for its simplicity and uniform architecture. It primarily consists of stacked convolutional layers with small 3×3 kernels followed by max-pooling and fully connected layers. Despite its relatively high parameter count, VGG serves as a strong baseline for image classification tasks and is useful for evaluating performance on dense and repetitive patterns.

- **ResNet (Residual Network):** ResNet introduces residual connections that allow gradients to flow more effectively through very deep networks. This innovation enables the construction of extremely deep models without suffering from vanishing gradients. Its architecture contains residual blocks, each of which includes skip connections that bypass one or more layers. ResNet is widely used in both academic and industrial applications due to its robust performance and efficient training behavior.
- **MobileNet:** MobileNet is a lightweight neural network architecture optimized for mobile and embedded devices. It achieves efficiency by using depthwise separable convolutions, which reduce both the computational cost and the number of parameters. MobileNet is particularly suitable for performance benchmarking in low-resource environments, and it offers a useful contrast to the heavier VGG and ResNet models.

The composition of each model in terms of Linalg operation is detailed in table 5.4.

Model	**Total Ops**	`conv_2d`	`pooling`	`matmul`	`fill`	`generic`	`unknown`
MobileNetV2	524	35	1	1	21	448	18
ResNet	510	53	2	1	15	438	1
VGG	65	13	6	3	8	19	16

Table 5.4: Operational composition of the benchmarked neural network models.

The following table presents the speedups achieved by our reinforcement learning (RL) based optimization approach in comparison to standard PyTorch execution and PyTorch JIT.

NN Model	**Our RL**	**PyTorch**	**PyTorch JIT**
ResNet	2.1	**4.25**	4.23
MobileNet	**2.5**	0.41	0.39
VGG	1.09	**51.81**	51.52

Table 5.5: Speedups for different models using our RL-based approach, standard PyTorch execution, and PyTorch JIT (with latest optimizations).

For **ResNet**, the highest speedup is achieved by the standard PyTorch implementation (4.25x), slightly outperforming the PyTorch JIT version (4.23x). Our RL-based approach achieves a moderate speedup of 2.1x, indicating that while the agent found some effective schedules, it did not match the highly optimized static backend used by PyTorch for this architecture.

In contrast, for **MobileNet**, our RL approach significantly outperforms both PyTorch (0.41x) and PyTorch JIT (0.39x), achieving a speedup of 2.5x. This suggests that the agent is particularly effective in identifying optimizations for lightweight, mobile-friendly models that may not be well-targeted by traditional compilation heuristics.

For **VGG**, standard PyTorch and PyTorch JIT both deliver very high speedups (above 51x), which are likely due to backend-level kernel fusions and other low-level optimizations. Our RL-based approach achieves only a 1.09x speedup, indicating that the agent was not able to match the aggressive optimizations applied by the PyTorch backends in this case.

Overall, the results highlight that while our RL-based approach can yield competitive or superior performance in some cases but it may not yet match the maturity and specialization of existing compiler optimizations for highly-tuned architectures like VGG and ResNet.

Interpretation

The performance results reveal a notable difference between our agent's optimization strategies and the highly-tuned heuristics of mature compilers. While the agent demonstrates a clear ability to find effective schedules—evidenced by the 2.5x speedup on MobileNetV2—it struggles to compete with established backends on more traditional architectures like VGG. We identify two primary factors contributing to this performance gap, both directly linked to the composition of the models detailed in Table 5.4.

First, a **limited action space and un-optimizable operations**. The set of transformations available to our agent is a subset of those in a full-fledged compiler. More importantly, our agent cannot optimize operations classified as `unknown`. This limitation is particularly impactful for VGG. As shown in Table 5.4, 16 of VGG's 65 total operations (nearly 25%) are `unknown`. This means a significant portion of the model is fundamentally outside the scope of our agent's optimization capabilities, severely handicapping its potential for speedup. In contrast, `unknown` operations constitute a negligible fraction of the much larger ResNet and MobileNetV2 models.

Second, a **trade-off between model scale and optimization strategy**. The scale of the model appears to dictate the most effective optimization approach.

- For **VGG**, its small size (65 operations) and regular, repetitive structure make it an ideal target for specialized, hand-crafted compiler rules. The phenomenal 51x speedup achieved by PyTorch strongly suggests the use of aggressive, hard-coded kernel fusions and low-level optimizations perfected for this common pattern. A generalist learning agent is at a disadvantage against such a specialized solution.
- For **MobileNetV2 and ResNet**, their larger scale (over 500 operations) and architectural complexity make it more difficult to apply simple, hand-tuned heuristics across the entire graph. This complexity creates a larger search space with more opportunities for optimization, which is where our RL agent excels. By exploring non-obvious combinations of transformations, the agent can discover effective schedules—like for MobileNetV2—that may be missed by static, pattern-based approaches. However, the agent's limited training time for exploration still prevents it from matching the highly refined optimizations for ResNet.

In summary, our agent shows promise on large, complex models where manual tuning is difficult, but it is constrained by its action space and exploration budget. These limitations frame clear directions for future work: expanding the agent's set of transformation primitives to cover more operations and developing more efficient exploration strategies to unlock complex, long-horizon optimizations.

Conclusion

Our evaluation on full neural network models reveals that our RL agent's effectiveness is strongly dependent on model complexity and composition. The agent delivers competitive speedups on large, intricate models like MobileNetV2, where it can successfully navigate a vast search space that is challenging for static heuristics. Conversely, it struggles on smaller, regular architectures like VGG, which benefit from years of specialized, hand-tuned compiler optimizations and contain a significant portion of operations our agent cannot yet optimize. This positions our learning-based method as a powerful tool for novel or less-common architectures, while highlighting that future work must focus on expanding the agent's action space and improving search efficiency to rival the performance of mature compilers across all domains.

General Conclusion

In this work, we extended a reinforcement learning (RL) agent to support the optimization of full neural network code. We began by updating the agent's state and observation representations through the integration of abstract syntax tree (AST) information extracted from Linalg operations. Additionally, we expanded the action space by introducing new actions such as `next` and `fusion`, enabling the agent to effectively navigate and optimize entire neural network computation graphs.

We also constructed a new dataset composed of single operations, synthesized sequences of operations, and modules extracted from real-world models such as ResNet. This allowed us to train the agent on a diverse and representative set of inputs. To support this new dataset, we updated the environment logic to ensure compatibility and enable the agent to traverse and interact with the operations correctly.

To validate the agent's ability to learn specific transformations, we first trained it on simple blocks of linear operations, where it successfully learned to apply the fusion transformation. We then extended training to the full dataset and evaluated the agent on complete neural network models. The evaluation showed promising results—demonstrating that the agent was able to find effective optimization schedules and, in some cases, even outperform traditional compilers like PyTorch.

In conclusion, while the results are encouraging, several challenges remain. Future work can focus on improving the agent's performance through the following directions:

- **Parallelizing training across multiple computational nodes**, allowing the agent to explore a significantly larger portion of the search space within a reasonable time budget. This would enable the discovery of more complex and effective optimization strategies.
- **Expanding the action space** to include additional optimization techniques such as loop skewing and unrolling, thereby offering the agent a richer set of transformations to apply.
- **Replacing the LSTM-based encoder** with more powerful sequence modeling architectures, such as attention mechanisms or transformer networks, which may provide better representations of the operation sequences and improve decision-making.

Bibliography

Aho, Alfred V. et al. (2006). *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA: Pearson Education. ISBN: 978-0321486813.

Bacon, David F., Susan L. Graham, and Oliver J. Sharp (1993). "Compiler transformations for high-performance computing". In: *EECS Department, University of California, Berkeley* UCB/CSD-93-781. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/1993/6309.html.

Baghdadi, Ramzi et al. (2018). "Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code". In: *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*.

Bendib, Nazim, Iheb Nassim Aouadj, and Riyadh Baghdadi (2024). *A Reinforcement Learning Environment for Automatic Code Optimization in the MLIR Compiler*. arXiv: 2409.11068 [cs.LG]. URL: https://arxiv.org/abs/2409.11068.

Berner, Christopher et al. (2019). "Dota 2 with Large Scale Deep Reinforcement Learning". In: *arXiv preprint arXiv:1912.06680*.

Bhattacharjee, Shatanik (2024). *Static vs. dynamic code analysis: What's the difference?* URL: https://vfunction.com/blog/static-vs-dynamic-code-analysis/.

Bishop, Christopher M (2006). *Pattern Recognition and Machine Learning*. Springer. URL: https://link.springer.com/book/9780387310732.

Brauckmann, Alexander, Andrés Goens, and Jeronimo Castrillon (2021). "PolyGym: Polyhedral Optimizations as an Environment for Reinforcement Learning". In: *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 17–29. DOI: 10.1109/PACT52795.2021.00009.

Cummins, Chris et al. (2021). "CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research". In: *arXiv preprint arXiv:2109.08267*. URL: https://arxiv.org/abs/2109.08267.

Fog, Agner (1996). *Optimizing Assembly Code*. URL: https://www.agner.org/optimize/optimizing%5C_assembly.pdf.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer. URL: https://link.springer.com/book/10.1007/978-0-387-84858-7.

He, Kaiming et al. (2016). "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pp. 770–778.

Hennouni, Et El Hassane et al. (2022). "Utilisation de l'apprentissage par renforcement dans l'exploration de l'espace de recherche pour l'optimisation automatique des compilateurs". In: *ResearchGate*. URL: https://www.researchgate.net/publication/362188197_Utilisation_de_l%27apprentissage_par_renforcement_dans_l%27exploration_de_l%27espace_de_recherche_pour_l%27optimisation_automatique_des_compilateurs.

Hessel, Matteo et al. (2018). "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*.

Laforest, Eric (2010). *Survey of Loop Transformation Techniques*. URL: https://fpgacpu.ca/writings/SurveyLoopTransformations.pdf.

Lamouri, Djamel and Djihane Merad (2024). *Utilisation d'apprentissage par renforcement pour l'optimisation automatique de code dans Tiramisu*. Available at: https://www.researchgate.net/publication/372128690_Utilisation_d'apprentissage_par_renforcement_pour_l'optimisation_automatique_de_code_dans_Tiramisu?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6InByb2ZpbGUiLCJwYWdlIjoicHJvZmlsZSJ9fQ.

Lattner, C. et al. (2021). "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation". In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

Lattner, Chris and Vikram Adve (2004). "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation". In: *Proceedings of the International Symposium on Code Generation and Optimization*. IEEE, pp. 75–86.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep learning". In: *Nature* 521.7553, pp. 436–444.

Mitchell, Tom M (1997). *Machine Learning*. McGraw-Hill. URL: https://www.cs.cmu.edu/~tom/mlbook.html.

Mnih, Volodymyr et al. (2015). "Human-level control through deep reinforcement learning". In: *Nature* 518.7540, pp. 529–533.

Mnih, Volodymyr et al. (2016). "Asynchronous methods for deep reinforcement learning". In: *Proceedings of the 33rd International Conference on Machine Learning* 48, pp. 1928–1937.

Neptune.ai (2024). *Neptune: Experiment tracking and model registry for MLOps*. https://neptune.ai. Accessed: 2025-06-22.

Paszke, Adam et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc.

Pecenin, Gustavo et al. (2019). "Learning to Optimize Halide with Deep Reinforcement Learning". In: *Proceedings of the Workshop on Machine Learning for Systems (MLSys)*.

Ragan-Kelley, Jonathan et al. (2013). "Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines". In: *ACM SIGPLAN Notices*. Vol. 48. ACM, pp. 519–530.

Schulman, John et al. (2017). "Proximal Policy Optimization Algorithms". In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, pp. 113–120.

Silver, David et al. (2016). "Mastering the game of Go with deep neural networks and tree search". In: *Nature* 529.7587, pp. 484–489.

Sutton, Richard S and Andrew G Barto (2018). *Reinforcement Learning: An Introduction*. MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Sutton, Richard S. et al. (1999). "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems (NIPS)*.

Van Hasselt, Hado, Arthur Guez, and David Silver (2016). "Deep reinforcement learning with double Q-learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*.

Wang, Yu, Hongyu Chen, and Ke Wang (2024). "Beyond the Phase Ordering Problem: Finding the Globally Optimal Code wrt Optimization Phases". In: *arXiv preprint arXiv:2410.03120*. URL: https://arxiv.org/abs/2410.03120v2.

Watkins, Christopher JCH and Peter Dayan (1992). "Q-learning". In: *Machine Learning* 8.3-4, pp. 279–292.

Williams, Ronald J. (1992). "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8.3-4, pp. 229–256.